

Towards Efficient Federated Recommendation with Discrete Graph Convolutional Network

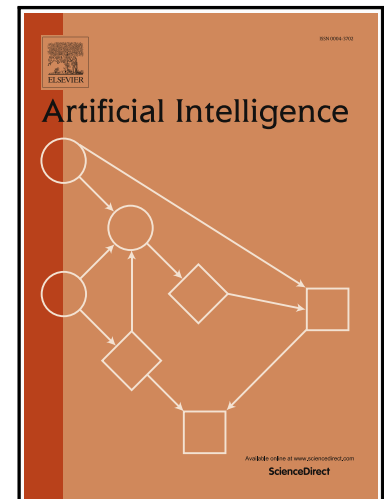
Yang Li, Weike Pan, Qiang Yang, Zhong Ming

PII: S0004-3702(26)00133-5
DOI: <https://doi.org/10.1016/j.artint.2026.104607>
Reference: ARTINT 104607

To appear in: *Artificial Intelligence*

Received date: 8 January 2026
Revised date: 24 June 2026
Accepted date: 26 August 2026

Please cite this article as: Yang Li, Weike Pan, Qiang Yang, Zhong Ming, Towards Efficient Federated Recommendation with Discrete Graph Convolutional Network, *Artificial Intelligence* (2025), doi: <https://doi.org/10.1016/j.artint.2026.104607>



This is a PDF of an article that has undergone enhancements after acceptance, such as the addition of a cover page and metadata, and formatting for readability. This version will undergo additional copyediting, typesetting and review before it is published in its final form. As such, this version is no longer the Accepted Manuscript, but it is not yet the definitive Version of Record; we are providing this early version to give early visibility of the article. Please note that Elsevier's sharing policy for the Published Journal Article applies to this version, see: <https://www.elsevier.com/about/policies-and-standards/sharing#4-published-journal-article>. Please also note that, during the production process, errors may be discovered which could affect the content, and all legal disclaimers that apply to the journal pertain.

Towards Efficient Federated Recommendation with Discrete Graph Convolutional Network

Yang Li^a, Weike Pan^{a,*}, Qiang Yang^b, Zhong Ming^c

^a*College of Computer Science and Software Engineering, Shenzhen University, Shenzhen, China*

^b*Academy for Artificial Intelligence, Hong Kong Polytechnic University, Hong Kong, China*

^c*Guangdong Laboratory of Artificial Intelligence and Digital Economy (SZ), Shenzhen, China*

Abstract

Federated graph convolutional network (FedGCN) is an emerging artificial intelligence technique in recommender systems, which allows each user to hold his or her own user-item bipartite subgraph and capture the high-order information in a [non-existent global graph](#) through collaborative training. However, the existing works on FedGCN are based on a [full-precision](#) continuous embedding space, which leads to the high overhead of communication and computation at the clients. In addition, the constructed global graph is usually expensive and suboptimal, and how to construct an accurate and privacy-preserving global graph at a low cost is still a challenge. To address these challenges, we propose a novel privacy-preserving recommendation method, i.e., federated hash graph convolutional network for recommendation (FedHashRec). Specifically, we first design a new hash graph convolutional method (HashRec) to map the [full-precision](#) embeddings into the Hamming space and then learn the similarity between users and items. Secondly, we design a privacy-preserving framework with triple protector (TriPro) to construct a fake global graph in the server through efficient client-client and client-server communication. Each item hash code is updated by obtaining the information of the fake user-item bipartite graph to avoid frequent communication. Moreover, we provide a rigorous the-

*Corresponding authors.

Email addresses: liyang22@email.szu.edu.cn (Yang Li), panweike@szu.edu.cn (Weike Pan), qyang@cse.ust.hk (Qiang Yang), mingz@szu.edu.cn (Zhong Ming)

oretical proof to show that the information obtained by the server from this fake user-item bipartite graph is lossless. We conduct extensive experiments on four real-world datasets to demonstrate that our method outperforms the existing **fully discrete** methods in terms of accuracy and efficiency. The datasets, source codes and scripts are available at <https://github.com/YannickCodeHome/FedHashRec>.

Keywords: One-Class Collaborative Filtering, Learning to Hash, Federated Learning, Graph Convolutional Network

1. Introduction

The recommender systems model the user-item historical data to mine the potential preferences of users and predict accurate users' tastes. Recently, network embedding methods [1, 2, 3], known for their high recall, have been extensively studied in recommender systems. Among them, graph convolutional network (GCN) [4, 5, 6] is one of the most widely used recommendation technique, which usually captures high-order information in the user-item bipartite graph by using neighborhood aggregation and message passing mechanisms to enhance the representations of the continuous embeddings of the nodes and improve the recommendation performance. In general, the raw data (user-item bipartite graph) is kept in the server for model training, which is exposed and raises serious privacy concerns. With the constraints of increasingly strict data protection laws and regulations, such as the general data protection regulation (GDPR)¹, platforms providing personalized services are not allowed to collect and use users' private data without consent, which leads to the occurrence of data isolation.

Federated learning (FL) [7, 8, 9, 10] provides a feasible solution to deal with privacy leakage, which allows a model to be trained without the raw data. The combination of FL and GCN, i.e., FedGCN, is considered as an emerging research topic for building privacy-preserving recommender systems, which reflects the trend of the evolution of artificial intelligence technology to privacy protection. **In federated recommendation, the global graph does not exist and the training of the global model is done through complex client-server collaboration.** However, to the best of our knowledge, more

¹<https://gdpr-info.eu>.

existing works [11, 12, 13] based on FedGCN aim to improving model performance with **full-precision** modeling, which face the following two challenges. (i) High resource consumption. In distributed scenarios, neighborhood aggregation requires frequent communication between the server and clients for high-dimensional and dense continuous embeddings, resulting in huge communication bandwidth consumption [14, 15]. In addition, for a client with many neighbors, training a local GCN model using the **full-precision** continuous embeddings is very expensive, which may be unaffordable for some clients. (ii) Hard to build a global user-item bipartite graph. Under the premise of privacy protection, how to securely and efficiently construct an accurate global user-item bipartite graph (**which does not leak privacy**) based on the isolated first-order subgraphs of the clients is still an open problem.

Learning to hash [16, 17, 18, 19] is introduced to recommender systems to overcome the high overhead of computation, which makes us inspired. The basic idea of learning to hash is to construct a mapping function that maps high-dimensional continuous embeddings into the Hamming space and generates **hash codes** (usually binary hash codes). In this case, learning to hash accelerates the inference process by transforming the original complex similarity calculation into efficient retrieval in the Hamming space. However, it brings the difficult problem of model optimization [20]. Since this mapping function is usually an infeasible differentiable discrete function, the gradient of this function is 0 almost at all the different values of the independent variable, resulting in the gradient to vanish in backpropagation.

Some existing works [21, 22, 23] introduce two solutions, which are the proxy function and approximation strategy. The first scheme that introduces the proxy function divides model training into two phases, namely training and inference. In the training phase, the mapping function is the proxy function, which is differentiable and similar to the discrete function. For example, HS-GCN [21] uses the hyperbolic tangent function $\tanh$ instead of the sign function for model optimization. In the inference phase, the mapping function uses the sign function to obtain the discrete hash codes to predict users' tastes. However, this scheme defeats the fundamental purpose of applying learning to hash to recommender systems, which is to improve the efficiency of model training. In the training phase, there is more than 90% of the time consumption of the whole recommender system, while the inference phase only takes a small part of it, which needs the binary inner product (XOR operation). The second scheme that introduces an approximation strategy aims to use an approximate gradient in backpropagation instead

of the derivative of the discrete function to avoid the problem of vanishing gradients. In this scheme, both the training phase and the inference phase are in the Hamming space, only the problem of vanishing gradients needs to be dealt. For example, HashGNN [22] introduces the straight through estimator (STE) to obtain the approximate gradient, which is a widely used approximation strategy when gradient descent is used to train a discrete model. In this paper, we choose the second scheme and introduce STE to solve the problem of vanishing gradients.

To address the two challenges mentioned before, we propose a novel privacy-preserving method, i.e., federated hash GCN for recommendation (FedHashRec). For better understanding, we describe our FedHashRec from two parts, namely the graph convolutional method and the federation technique.

We first introduce the graph convolutional method we designed, i.e., HashRec. Specifically, we divide the modeling process of our HashRec into three components, namely the initialization layer, the propagation layer and the prediction layer. It should be noted that different from the existing works [24, 25, 22], in order to better represent the semantics of hash codes, we map the continuous embeddings into the ternary Hamming space instead of the binary Hamming space. In the initialization layer, we introduce a discrete function sign to map the embeddings into the Hamming space to ensure that the input of the first convolutional layer is discretized. In the propagation layer, we design a convolution rule to learn the high-order information in the user-item bipartite graph. The key of this convolution rule is that the update of each node's hash code depends on the dominant signal transmitted by the corresponding neighbors, as well as the credibility weight of the opinion agreement of these neighbors. More importantly, we need to discretize each output, ensuring that the hash codes generated by the intermediate convolutional layers are discrete. In the prediction layer, we define the net alignment strength of the hash codes of user u and item i , which is the difference between the number of positive and negative agreement signals, as the similarity between user u and item i . To guarantee the feasibility of the optimization, in backpropagation, we introduce STE to solve the problem where the gradient of the discrete function is 0. By stacking the propagation layers, we can learn the high-order information of the user-item bipartite graph.

Then, we introduce the federation technique, i.e., TriPro, which is a secret sharing framework with triple protector. We find that there is a risk of privacy sensitive data leakage only when the server learns the l -layer hash

code of item i . Wherein the privacy sensitive data is the hash code of each user u and the number of users who have interacted with item i , denoted as $\mathbf{h}_u^{(l)}$ and $|\mathcal{U}_i|$. To protect these data, we divide the secret of each user into three function zones: adaptation zone, secret zone and verification zone. Specifically, the adaption zone keeps some items of user u , which include all positive items $\mathcal{I}_u$ and some negative items $\mathcal{I}'_u$ to confuse the malicious attackers. The secret zone keeps multiple segments, which are obtained by users splitting their own hash codes. The verification zone consists of some segments for each item, and each segment is composed of two parts, namely, the hash code verification segment and the item authenticity verification bit. In secret sharing, each user u randomly combines the segments of the secret zone and the verification zone, and then pack them into multiple secrets together with the adaptation zone. Then, each user keeps one of the secrets and sends the rest to other participants. Moreover, if user u receives a secret, he or she will merge the received secret with the existing secret by different zones to generate a fake secret. Finally, user u uploads the fake secret to the server, which is public. The server builds a fake global graph with all the users' fake secrets and extracts information from it to learn the hash code of each item. By doing this, each user only needs to perform secret sharing once in each graph convolution operation. In addition, we theoretically prove that the information extracted by the server is lossless.

We summarize our main contributions as follows.

- We propose a novel privacy-preserving hashing- and GCN-based recommendation method, i.e., federated hash GCN for recommendation (FedHashRec). To the best of our knowledge, this is the first attempt to model the Hamming similarity based on the high-order information between users and items in FedGCN with a fully discrete method.
- We design a graph convolutional method in FedHashRec, i.e., HashRec. We specifically make the following efforts: (i) we design a novel graph convolution scheme based on full discretization in the Hamming space for capturing the Hamming similarities based on the high-order information between users and items according to the correlation between nodes; (ii) we introduce a novel sign constrained loss as a term for multi-objective optimization, which better constrains the update direction of the hash code; and (iii) we give the mathematical theoretical analysis of backpropagation, which shows that our designed graph convolution rule is effective.

- We propose a secret sharing framework with triple protector, i.e., TriPro. We put the privacy sensitive data into three functional zones to resist the attack of malicious users through triple protection, i.e., adaptation zone, secret zone and verification zone. And we only need to perform secret sharing once in each convolutional layer to obtain all the information required. Moreover, we theoretically prove that there is no information loss in TriPro.
- We conduct extensive experiments on four real-world datasets to demonstrate that our method outperforms the existing [fully discrete](#) methods in terms of accuracy and efficiency.

2. Related Work

2.1. Graph Convolutional Network for Recommendation

Graph convolutional network (GCN) has been widely used in recommender systems due to the powerful ability to model the high-order interactions among users and items by using the structural information of the user-item bipartite graph. PinSage [26] is an early work, which combines random walks and graph convolutions to generate embeddings of nodes that contain graph structure and node feature information, improving robustness and convergence of the model. NGCF [2] captures cooperative signals in high-order connections by stacking multiple embedding propagation layers. LightGCN [5] designs a lightweight graph convolution rule, which includes only the most essential component, i.e., neighborhood aggregation. GraphPro [27] merges dynamic graph pre-training with prompt learning in a parameter-efficient manner to improve the timeliness of user preferences. LightGNN [28] is a lightweight and distillation-based GNN pruning framework to enable precise pruning of misinformation. The aggregation strategies of these methods are all based on [real-valued embeddings](#), which are not directly applicable to the Hamming space. [In contrast, our FedHashRec introduces a fully discrete graph convolution rule for the Hamming space.](#)

2.2. Learning to Hash for Recommendation

Learning to hash provides a solution for large-scale recommendation scenarios. DCF [17], DFM [18], DPR [29] and MFDCE [30] learn user-item binary codes via discrete optimization. In this paper, we mainly focus on GCN-based recommendation methods. DGCN-BinCF [31] is the first work

that combines GCN and learning to hash to aggregate the high-order features of users and items. CIGAR [32] proposes a candidate generation and re-ranking based framework, which helps to efficiently generate high-quality candidates. HashGNN [22] introduces a GNN encoder and a hash layer, and improves the representation of hash codes by optimizing the cross-entropy loss and ranking loss. HS-GCN [21] designs a Hamming spatial graph convolutional network to model the Hamming similarity and embed it into the codes of users and items. BGCH [33] proposes adaptive graph convolutional hashing and latent feature dispersion to retain the structural information, and introduces Fourier series decomposition to estimate gradients. BGCH+ [23] is an improved algorithm of BGCH, which introduces a dual augmentation method to deal with intermediate information and hash code outputs in the latent feature space. GraphHash [34] uses modularity-based bipartite graph clustering to generate user/item bucket assignments to reduce embedding table sizes. To the best of our knowledge, no existing work has studied **fully discrete** GNN-based recommendation methods, which is the problem of our study. **Unlike these hashing methods that preserve the real-valued space during the training or inference phase, our FedHashRec strictly uses discrete hash codes throughout the training and inference phases, which retains the full advantage of the Hamming space.**

2.3. Federated Recommendation

Federated recommendation aims to develop a new recommendation framework that can make recommendations without privacy leakage. FCF [35] is the first proposed federated collaborative filtering algorithm, which allows users to keep private data locally and send updates to the server to update the global model. FedRec [36] uses the hybrid filling strategy and the way of sampling negative items for each user to train a federated recommendation model. FMSS [37] achieves lossless transmission of parameters by secret sharing. FedPerGNN [11] introduces the third-party server to expand subgraphs to learn high-order information of nodes, and introduces local negative sampling and local differential privacy to protect users' private data. D2PDRF [38] introduces an attention mechanism and differential privacy to improve the recommendation performance of the model by adjusting the blend of long-term and short-term interests. CoLR [39] introduces an alternative compression method to reduce the communication burden of the system, and introduces homomorphic encryption to protect data privacy. FedDAE [40] adopts decentralized architecture from the FL scheme, and in-

introduces a dual-encoder mechanism to separately preserve shared knowledge across clients and client-specific personalized knowledge. ProxyRL-FRS [41] introduces a reinforcement learning framework for federated client selection, using a proxy model to balance recommendation accuracy and embedding staleness. BalFed [42] introduces a federated rebalancing framework using gradient alignment regularization to mitigate model bias caused by data imbalance across clients. GFRec [43] uses a lightweight user grouping aggregation to enable collaborative filtering among users with similar preferences and is extended to signed recommendations. Among the existing works, there is no learning to hash method designed for federated recommendation. Our FedHashRec bridges this gap by combining federated recommendation and learning to hashing, where our method differs from existing methods in that the information of the communication between clients and between each client and the server is fully discrete.

3. Preliminaries

In this section, we first introduce Hamming space and Hamming similarity, and then we provide the problem definition. We list some notations and their explanations used in this paper in Table 1.

3.1. Hamming Space

Hamming space is a fundamental concept in coding theory, which is a better way to measure the distance between words over the alphabet. For better understanding, we give the mathematical definition inspired by [44].

Definition 1. (*Hamming Space*). A given Hamming space, denoted $\mathcal{H}(q, n)$, is the set of n -length words over the alphabet $\mathcal{X}_q$, where $\mathcal{X}_q = \{0, 1, \dots, q-1\}$. We have the formula of the Hamming space:

$$\mathcal{H}(q, n) = \{(x_1, x_2, \dots, x_n)\}, \quad (1)$$

where $(x_1, x_2, \dots, x_n) \in \mathcal{X}_q^n$ and the cardinality of this space is $|\mathcal{H}(q, n)| = q^n$.

In the existing works [24, 25], Hamming space is generally regarded as a binary discrete space, which is actually a special case of the Hamming space in $q = 2$, formally denoted as $\mathcal{H}(2, n)$. This design stems from the field of signal processing, which is a narrow mathematical view that is not conducive to promoting its range of applications. It should be noted that the alphabet

$\mathcal{X}_q$ is not restricted to numerical sets like $\{0, 1, \dots, q-1\}$ and it can be any finite set of distinct symbols. For example, in HS-GCN [21], the alphabet $\mathcal{X}_2 = \{-1, 1\}$.

3.2. Hamming Similarity

In learning to hash, in order to pursue efficient storage and fast similarity calculation, user- and item-latent vectors are often mapped into a Hamming space [17]. In this space, the original high-dimensional vectors of floating-point numbers are replaced by **binary hash codes**. For a given user u and item i , we denote their K -dimensional hash codes as $\mathbf{h}_u \in \{\pm 1\}^{1 \times K}$ and $\mathbf{h}_i \in \{\pm 1\}^{1 \times K}$, respectively. This discretization is the core feature of hash codes and allows them to perform efficient bit-wise operations. The similarity between user u and item i is represented by their Hamming similarity, which is defined as follows [21]:

$$S_{ui} = \frac{1}{K} \sum_{k=1}^K \mathbb{I}(h_{uk} = h_{ik}), \quad (2)$$

where h_{uk} and h_{ik} are the k -th bit of $\mathbf{h}_u$ and $\mathbf{h}_i$, and $\mathbb{I}$ denotes the indicator function that returns 1 if the statement is true and 0 otherwise.

We can measure the inherent agreement between user hash codes and item hash codes by calculating the sum of the products of all the corresponding bits:

$$D_{ui} = \mathbf{h}_u^\top \mathbf{h}_i = \sum_{k=1}^K h_{uk} \times h_{ik}. \quad (3)$$

Through analysis, we consider two boundary scenarios. In the first scenario, the hash codes are same at every bit position. In this case, each product term $h_{uk} \times h_{ik} = 1$, resulting in the maximum value $D_{ui} = K$. Conversely, when the hash codes are completely different, meaning that they differ at every bit. In this case, each product term $h_{uk} \times h_{ik} = -1$, resulting in the maximum value $D_{ui} = -K$. However, hash codes typically exhibit partial agreement, i.e., $\mathbf{h}_u$ and $\mathbf{h}_i$ have some same bits and different bits. We denote n is the number of the same bits, and $K - n$ as the number of the different bits. D_{ui} can be further denote as follows:

$$D_{ui} = n \times 1 + (K - n) \times (-1) = 2n - K. \quad (4)$$

We can further derive Eq.(2):

$$S_{ui} = \frac{1}{K} \sum_{k=1}^K \mathbb{I}(h_{uk} = h_{ik}) = \frac{n}{K} = \frac{1}{2} + \frac{1}{2K} \mathbf{h}_u^\top \mathbf{h}_i. \quad (5)$$

Based on the above proof, we know that $S_{ui} \propto \mathbf{h}_u^\top \mathbf{h}_i$ and we can use the inner product of $\mathbf{h}_u$ and $\mathbf{h}_i$ to represent the similarity between user u and item i .

3.3. Problem Definition

In a decentralized setting, there are a set of n users $\mathcal{U}$ and a set of m items $\mathcal{I}$. We use $\mathcal{G} = \{\mathcal{U}, \mathcal{I}, \mathcal{E}\}$ to denote the global user-item bipartite graph. Wherein each user holds his or her own interactions, forming a local subgraph $\mathcal{G}_u = \{\{u\}, \mathcal{I}_u, \mathcal{E}_u\}$, where $\mathcal{I}_u$ denotes the set of the interacted items by user u and $\mathcal{E}_u$ denotes the edge set connected to the node of user u . We use $\mathbf{R} \in \{0, 1\}^{n \times m}$ to denote the user-item interaction matrix, where $r_{ui} \in \mathbf{R}$. Specifically, each user u only knows his or her own row $\mathbf{R}_u$. For each user u , $r_{ui} = 1$ if $(u, i) \in \mathcal{E}_u$, and $r_{ui} = 0$ otherwise. Our goal is to train a discrete global model by the server only by coordinating collaborations across users without leaking any user's private interactions (e.g., $\mathcal{G}_u$) and the intermediate representations (e.g., hash code).

4. METHODOLOGY

Our FedHashRec consists of two components: the graph convolution method HashRec and the privacy-preserving framework TriPro. In this section, we first introduce our HashRec. Secondly, in order to better optimize the model, we introduce a multi-objective optimization paradigm. Next, we explain the key points in backpropagation, which help to better understand the advantages of our technique. Then, we detail our TriPro. Finally, we provide the complexity and privacy analysis of our FedHashRec.

4.1. HashRec

To address the first challenge mentioned in Section 1, we propose a graph convolutional method, i.e., HashRec. As shown in Figure 1, our HashRec consists of three components: initialization layer, propagation layer and prediction layer. In the initialization layer, the embeddings of users and items

Table 1: Some notations and explanations used in the paper.

Notation	Explanation
n	the number of all users
m	the number of all items
u	user ID
i	item ID
r_{ui}	observed rating of user u on item i
$\hat{r}_{ui}$	predicted rating of user u on item i
$\mathbf{R}$	the user-item interaction matrix
$\mathcal{R}^+ = (u, i)$	the observed (user, item) pairs in training data
$\mathcal{R}^- = (u, j)$	the unobserved (user, item) pairs in training data
N	the number of records, i.e., $ \mathcal{R} $
$\mathcal{U}$	the set of all users
$\mathcal{I}$	the set of all items
$\mathcal{I}_u$	the set of the interacted items by client u
$\mathcal{U}_i$	the set of the users who interact with item i
$\mathcal{I}'_u, \mathcal{I}'_u = \rho \mathcal{I}_u $	the set of the sampled non-interacted items w.r.t. client u from $\mathcal{I} \setminus \mathcal{I}_u$
$\mathbf{U}_u^{(l)} \in \mathbb{R}^{1 \times K}$	the l -th layer embedding of user u
$\mathbf{V}_i^{(l)} \in \mathbb{R}^{1 \times K}$	the l -th layer embedding of item i
$\mathbf{h}_u^{(l)} \in \mathbb{R}^{1 \times K}$	the l -th layer hash code of user u
$\mathbf{h}_i^{(l)} \in \mathbb{R}^{1 \times K}$	the l -th layer hash code of item i
K	the number of latent dimensions
ρ	the sampling parameter
γ	the learning rate
τ	the user batch size
λ_1	the trade-off parameter of the ranking loss
λ_2	the trade-off parameter of the sign constrained loss

are mapped into the Hamming space by a discrete function. In the propagation layer, according to the characteristics of the ternary hash codes, we design a new convolution rule to learn multi-order information. In the prediction layer, we give the prediction function to predict the users' tastes.

4.1.1. Initialization Layer

As mentioned in Section 1, we choose the approximation strategy as our discrete scheme, which can maximize the advantage of the Hamming space. Specifically, we map the embeddings of users and items into the Hamming space by directly taking the sign function as the discrete function in the initialization layer. The formula is as follows:

$$\mathbf{h}_u = \text{sign}(\mathbf{U}_u), \mathbf{h}_i = \text{sign}(\mathbf{V}_i), \quad (6)$$

where $\mathbf{U}_u \in \mathbb{R}^{1 \times K}$ and $\mathbf{V}_i \in \mathbb{R}^{1 \times K}$ denote the embedding of user u and item i .

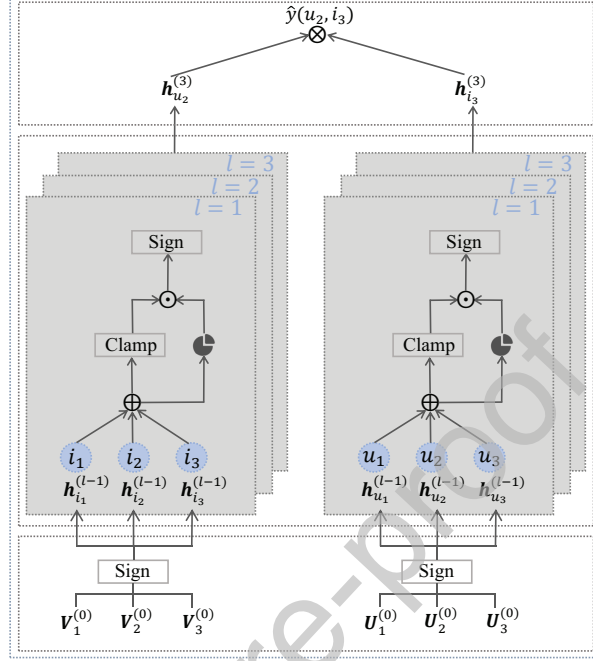


Figure 1: Illustration of our HashRec, which consists of three components. The initialization layer (bottom) discretizes the embeddings, the propagation layer (middle) integrates multi-order neighbor information, and the prediction layer (top) infers user tastes. In the propagation layer, the pie chart means calculating the credibility weight of a node signal. We assume that i_1, i_2, i_3 are the interacted items by user u_2 and u_1, u_2, u_3 are the users who interact with item i_3 .

In the existing works, all the hash codes are binary, which enforce that each bit of the hash codes has a certain signal (positive or negative). The design of the binary hash codes is contrary to reality. In a recommendation task, there are uncertain preferences, i.e., neutral or missing information, which is inherent and persistent.

In order to represent the uncertain preferences, in this paper, we introduce the ternary Hamming space ($\{-1, 0, 1\}$) instead of the binary Hamming space ($\{-1, 1\}$), where the 0 state in the ternary hash code means uncertain preferences. Compared with the binary hash code, the ternary hash code consumes an extra bit, but it can capture more accurate information, i.e., semantic nuances, user uncertainty and sparse interaction patterns, which is the core point in the recommendation task.

For clarity, we introduce the sign function that maps the embeddings of users and items to the ternary Hamming space:

$$\text{sign}(x) = \begin{cases} -1, & \text{if } x < 0 \\ 0, & \text{if } x = 0 \\ 1, & \text{if } x > 0 \end{cases} \quad (7)$$

Through the initialization layer, we will use hash codes instead of real-valued embeddings in the training phase, which not only improves the computational efficiency of the training phase, but also reduces the communication bandwidth consumption by $16\times$ (a floating-point number requires 32 bits and each value of a hash code requires 2 bits).

4.1.2. Propagation Layer

Neighborhood Aggregation. Graph-based recommendation models the interactions between users and items (e.g., clicks, purchases, ratings, etc.) as a user-item bipartite graph. Since the behavior of a node is often influenced by its neighbors, how to effectively aggregate the information of the neighbors and generate more accurate user and item embeddings becomes crucial. In the traditional GCN, the server aggregates neighbor information through the weighted average of high-dimensional floating-point embedding. Taking LightGCN as an example, it aggregates neighbor information by introducing the normalization of node degrees as weights, which is in the real-valued space.

In this paper, we replace real-valued embeddings with the ternary hash codes. In this case, the traditional aggregation strategy is infeasible, since otherwise it will destroy the discrete nature of the hash code. Hence, we design a convolution rule that is suitable for the ternary hash codes. The basic strategy is to aggregate messages in the neighborhood of the central node. We can obtain the signal of the interactions of the user node u of the l -th propagation layer:

$$\mathbf{m}_u^{(l)} = \sum_{i \in \mathcal{I}_u} \mathbf{h}_i^{(l)}, \quad (8)$$

where $\mathcal{I}_u$ is the set of the rated items by user u and $\mathbf{h}_i \in \{-1, 0, 1\}^K$. Note that in this section we only describe the convolution process at the user node, since the convolution process at the item node is similar to that at the user node.

At the l -th propagation layer, the numerical range of the signal $\mathbf{m}_u^{(l)}$ is significantly affected by the number of neighbors of user u . Active users with large $\mathcal{I}_u$ accumulate stronger signals, resulting in huge differences from cold start users (fewer interactions), which is not conducive to the stability of the model. Therefore, we need to further process it.

Message Processing. The difference in signal amplitude between active and cold start users is up to a hundred times, resulting in a gradient imbalance during model training. To eliminate the direct effect of the number of neighbors on the message amplitude, we average the signal and convert the original accumulated signal to the average signal strength. We can obtain the following formula:

$$\mathbf{p}_u^{(l)} = \frac{\mathbf{m}_u^{(l)}}{|\mathcal{I}_u|}, \quad (9)$$

where $|\mathcal{I}_u|$ is the number of the interacted items by client u and each bit of $\mathbf{p}_u^{(l)}$ is in the range $[-1, 1]$.

Each bit of $\mathbf{p}_u^{(l)}$ reflects the net tendency of neighbors toward +1 or -1 in that node u . Specifically, if the value of the k -th bit of $\mathbf{p}_u^{(l)}$ approximately approaches +1, most of the k -th bits of the hash codes of the neighbors representing node u are 1, and if the value of the k -th bit of $\mathbf{p}_u^{(l)}$ approximately approaches -1, most of the k -th bits of the hash codes of the neighbors representing node u are -1. In particular, if the value of the k -th bit of $\mathbf{p}_u^{(l)}$ approaches 0, it means that the neighbors of the node u have scattered opinions or weak signals at the k -th bit.

By analyzing the average signal strength, we find that its numerical distribution implies the agreement credibility of neighbor opinions. When neighbors agree, the value of the k -th bit of $|\mathbf{p}_u^{(l)}|$ tends to 1, which indicates strong credibility. Conversely, when there is a clear disagreement of neighbor opinions, the value of the k -th bit of $|\mathbf{p}_u^{(l)}|$ tends to 0, which indicates low credibility. Moreover, when the value of the k -th bit of $\mathbf{p}_u^{(l)}$ is equal to 0, it cannot directly act on the embedding update of the node, which is due to the easy cause of [vanishing gradients](#) during backpropagation. Therefore, we introduce the credibility weight $\mathbf{d}$, which represents the credibility of neighbor opinion agreement and will modulate the signal strength in the final embedding update. The formula of the credibility weight is as follows:

$$\mathbf{d}_u^{(l)} = \frac{1 + |\mathbf{p}_u^{(l)}|}{2}, \quad (10)$$

where $\mathbf{d}_u^{(l)}$ denote the l -th layer credibility weight of user u and the range of each bit is $[0.5, 1]$.

In addition, we also need to clamp $\mathbf{m}_u^{(l)}$ to extract the direction of the signal of the interactions of user u . If the value of the direction is 1, it means that there is a significant positive signal, if the value is -1, it means that there is a significant negative signal, and if the value is 0, it means that the signal is unclear, which needs be retained. Moreover, the clamp function can also constrain the range of values to prevent gradient exploding in backpropagation. The clamp function is expressed as follows:

$$\begin{aligned} \mathbf{s}_u^{(l)} &= \text{clamp}(\mathbf{m}_u^{(l)}) \\ &= \begin{cases} 1, & \text{if } \mathbf{m}_u^{(l)} > 1 \\ \mathbf{m}_u^{(l)}, & \text{if } -1 \leq \mathbf{m}_u^{(l)} \leq 1 \\ -1, & \text{if } \mathbf{m}_u^{(l)} < -1 \end{cases} \end{aligned} \quad (11)$$

Hash Code Update. We fuse information by element-wise multiplication of $\mathbf{s}_u^{(l)}$ and $\mathbf{d}_u^{(l)}$. Specifically, $\mathbf{s}_u^{(l)}$ provides the dominant signal of neighbor information, and $\mathbf{d}_u^{(l)}$ provides the credibility weight of neighbor opinion agreement on the corresponding dimension. Finally, we use the sign function to force transform the fused real-valued vectors back into the three-valued embedding space to preserve the discreteness of the embeddings. We have the update function:

$$\mathbf{h}_u^{(l+1)} = \text{sign}(\mathbf{s}_u^{(l)} \odot \mathbf{d}_u^{(l)}). \quad (12)$$

Such a fusion strategy makes the model optimize in the direction of high credibility and strong positive signal in the process of backpropagation.

More importantly, in our method, we can ensure that the ternary hash codes of all the nodes preserve the ternary property and never deteriorate into binary hash codes during the process of training. Firstly, the 0 state is inherent and persistent in recommender systems due to its special semantics, which is not permanently disappeared in the ternary hash codes. Secondly, our graph convolution rule states that as long as the neighbors of a specific node have the same amount of positive and negative signals, the aggregated result is exactly 0, which passes through and becomes the final 0 in the hash code of this node. Even if the hash code of a node temporarily deteriorates to a binary hash code in a training step (from ternary to binary), the subsequent graph convolutional training is most likely to make some values of the hash

code become 0 (from binary to ternary), since the node receives the same amount of positive and negative signals in some dimensions.

4.1.3. Prediction Layer

Since the deep propagation of graph convolution is able to integrate multi-order neighbor information, we take the hash codes at the last layer L as the representations of user u and item i to predict their interaction, formally,

$$\mathbf{h}_u^* = \mathbf{h}_u^{(L)}, \mathbf{h}_i^* = \mathbf{h}_i^{(L)}. \quad (13)$$

Unlike the binary hash codes, the ternary hash codes have some zero values. Each zero value in a ternary hash code has the key neutral semantics, which represents the lack of clear signal or disagreement of the hash code on this dimension. Therefore, it should be deemed invalid and does not contribute to positive or negative similarity to avoid false recommendations. We imitate the Hamming similarity definition for binary spaces to derive the Hamming similarity representation for ternary spaces and obtain the following formula,

$$S_{ui} = \frac{1}{2} - \frac{x}{2K} + \frac{1}{2K} \mathbf{h}_u^\top \mathbf{h}_i, \quad (14)$$

where x is the number of dimensions with value 0 in the two hash codes (i.e., $\mathbf{h}_u$ and $\mathbf{h}_i$) and K is the dimension of these hash codes.

By Eq.(14), we cannot conclude that $S_{ui} \propto \mathbf{h}_u^\top \mathbf{h}_i$, which is due to x is a variable. However, in the ternary hash space, we find that users and items with higher similarity have more net alignment strength in their non-zero hash dimensions. Specifically, the matching non-zero bits (both +1 or both -1) accumulate positive agreement signals, denoted by n^+ , while the opposing non-zero bits (one +1 and one -1) reduce positive agreement signals, denoted by n^- . We define the net alignment strength D as the difference between the number of positive and negative agreement signals,

$$D = n^+ - n^-. \quad (15)$$

This net alignment strength is mathematically quantified by the dot product, and then we can derive the prediction function:

$$\hat{r}_{ui} = \mathbf{h}_u^{*\top} \mathbf{h}_i^*. \quad (16)$$

4.2. Multi-Objective Optimization

In order to train the model to meet the requirements of recommendation accuracy and hash code learning simultaneously, we design a multi-objective loss function, which consists of three components, i.e., binary cross-entropy loss $\mathcal{L}_{\text{bce}}$, pairwise ranking loss $\mathcal{L}_{\text{rank}}$ and sign constrained loss $\mathcal{L}_{\text{sign}}$.

4.2.1. Binary Cross-entropy Loss

Binary cross-entropy loss measures the difference between the probability of the interaction $\sigma(\hat{r}_{ui})$ predicted by the model and the true observed interaction label r_{ui} (1 for interaction and 0 for no interaction). By minimizing this loss, the predicted similarity of observed interactions (positive samples) is made as high as possible, and the predicted similarity of unobserved interactions (negative samples) is made as low as possible. We implement $\mathcal{L}_{\text{bce}}$ with the following formula:

$$\mathcal{L}_{\text{bce}} = - \sum_{r_{ui} \in \mathbf{R}} r_{ui} \log(\sigma(\hat{r}_{ui})) + (1 - r_{ui}) \log(1 - \sigma(\hat{r}_{ui})), \quad (17)$$

where $\mathbf{R}$ is the user-item interaction matrix.

4.2.2. Pair wise Ranking Loss

In general, since user preferences often exhibit relative ranking relationships in recommender systems, we introduce the pairwise ranking loss. Specifically, for a given user u , we construct triples (u, i, j) , where item i is the observed interaction of user u and item j is the unobserved interaction of user u . The loss encourages observed interactions to have higher estimated values than unobserved interactions. In addition, it is necessary to enforce that at least a preset boundary value α is maintained between the two. Inspired by the above, we formulate $\mathcal{L}_{\text{rank}}$ in the form of hinge loss:

$$\mathcal{L}_{\text{rank}} = \sum_{(u,i,j) \in \mathcal{D}} \max(0, -\sigma(\hat{r}_{ui}) + \sigma(\hat{r}_{uj}) + \alpha), \quad (18)$$

where $\mathcal{D}$ is a triplet sampled based on the user-item interaction matrix $\mathbf{R}$. It should be noted that $(u, i) \in \mathcal{R}^+$ and $(u, j) \in \mathcal{R}^-$, where $\mathcal{R}^+$ indicates the observed interactions, and $\mathcal{R}^-$ is the unobserved interactions.

4.2.3. Sign Constrained Loss

In a traditional GCN for recommendation, the last term of the loss function is usually a regularization term, defined as the $\mathcal{L}_2$ norm of the initial embedding matrix $\mathbf{E}^{(0)}$, i.e., $\|\mathbf{E}^{(0)}\|_2^2$. Its significance is to prevent high-dimensional parameter overfitting, improve model generalization ability and maintain training stability by constraining the initial embeddings. This regularization term is effective in the continuous embedding recommendation scenario. However, in the task of hashing-based recommendation, this term fundamentally conflicts with our goal of mapping continuous embeddings into the Hamming space.

It should be emphasized that we accept the existence of zero values due to its special semantics. However, our final goal is to make each value in a hash code as 1 or -1 as possible. In our method, if we apply $\|\mathbf{E}^{(0)}\|_2^2$ as the regularization term, it will interfere with the model optimization direction and weaken the representation ability of the hash codes.

To solve this problem, we propose a sign constrained loss $\mathcal{L}_{\text{sign}}$ dedicated to the ternary Hamming space. By minimizing the squared deviation of the absolute value of the embedding from 1, we explicitly drive the value of each dimension of the embedding towards 1. In addition, it also inherits all the benefits of the regularization term $\|\mathbf{E}^{(0)}\|_2^2$. We define $\mathcal{L}_{\text{sign}}$ as follows,

$$\mathcal{L}_{\text{sign}} = \|\mathbf{1} - |\mathbf{E}^{(0)}|\|_2^2, \quad (19)$$

where $\mathbf{1}$ denotes a K -dimensional unit matrix and $|\cdot|$ denotes the absolute value of the input.

4.2.4. Optimization

By combining the considerations discussed above, the overall loss function to be minimized is formulated as:

$$\begin{aligned} \mathcal{L} &= \mathcal{L}_{\text{bce}} + \lambda_1 \mathcal{L}_{\text{rank}} + \lambda_2 \mathcal{L}_{\text{sign}} \\ &= - \sum_{r_{ui} \in \mathcal{R}} r_{ui} \log(\sigma(\hat{r}_{ui})) + (1 - r_{ui}) \log(1 - \sigma(\hat{r}_{ui})) \\ &\quad + \lambda_1 \sum_{(u,i,j) \in \mathcal{D}} \max(0, -\sigma(\hat{r}_{ui}) + \sigma(\hat{r}_{uj}) + \alpha) \\ &\quad + \lambda_2 \|\mathbf{1} - |\mathbf{E}^{(0)}|\|_2^2, \end{aligned} \quad (20)$$

where λ_1 and λ_2 are the trade-off parameter to balance the ratio between the binary cross-entropy loss $\mathcal{L}_{\text{bce}}$, the pairwise ranking loss $\mathcal{L}_{\text{rank}}$ and the sign

constrained loss $\mathcal{L}_{\text{sign}}$, $\mathbf{E}^{(0)} \in \mathbb{R}^{(N+M) \times K}$ denotes the embeddings of the 0-th layer, $|\cdot|$ means to obtain the absolute value and $\|\cdot\|_2$ denotes the $\mathcal{L}_2$ norm. We fix α to 0.2 in experiments. In order to solve the problem of vanishing gradients, we introduce the STE [22] as a key solution to solve the difficult discrete optimization problem. In the backward propagation process, we directly copy the gradient of the loss function on the hash code $\mathbf{h}$, which is $\frac{\partial \mathcal{L}}{\partial \mathbf{h}}$, to the gradient on the continuous latent representation $\mathbf{z}$, which is $\frac{\partial \mathcal{L}}{\partial \mathbf{z}}$.

This mechanism of forward hard quantization and backward soft estimation not only realizes the discretization of the embedding, but also improves the performance of the model through more refined gradient control.

4.3. Backpropagation Analysis

To further verify the effectiveness of our HashRec, we conduct an in-depth analysis of the theory of the backpropagation process. According to the forward propagation process of the model, taking the derivative of the loss function $\mathcal{L}$ with respect to the initial embedding $\mathbf{U}_u^{(0)}$ of user u as an example (the derivative for the initial embedding $\mathbf{V}_i^{(0)}$ of item i is similar to this one), we first establish the chain formula for the back propagation. This derivation process not only reflects the gradient flow in GCN, but also highlights the characteristics that alternately propagate between the user layer and the item layer. For clarity, we formulate the chain formula as follows:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial \mathbf{U}_u^{(0)}} &= \frac{\partial \mathcal{L}}{\partial \hat{r}_{ui}} \cdot \frac{\partial \hat{r}_{ui}}{\partial \mathbf{h}_u^{(L)}} \cdot \frac{\partial \mathbf{h}_u^{(L)}}{\partial \mathbf{h}_u^{(0)}} \cdot \frac{\partial \mathbf{h}_u^{(0)}}{\partial \mathbf{U}_u^{(0)}} \\ &= \frac{\partial \mathcal{L}}{\partial \hat{r}_{ui}} \cdot \frac{\partial \hat{r}_{ui}}{\partial \mathbf{h}_u^{(L)}} \cdot \left(\prod_{l=0}^{L-1} \frac{\partial \mathbf{h}_u^{(l+1)}}{\partial \mathbf{h}_u^{(l-1)}} \right) \cdot \frac{\partial \mathbf{h}_u^{(0)}}{\partial \mathbf{U}_u^{(0)}} \\ &= \frac{\partial \mathcal{L}}{\partial \hat{r}_{ui}} \cdot \frac{\partial \hat{r}_{ui}}{\partial \mathbf{h}_u^{(L)}} \cdot \left(\prod_{l=0}^{L-1} \frac{\partial \mathbf{h}_u^{(l+1)}}{\partial \mathbf{h}_i^{(l)}} \cdot \frac{\partial \mathbf{h}_i^{(l)}}{\partial \mathbf{h}_u^{(l-1)}} \right) \cdot \frac{\partial \mathbf{h}_u^{(0)}}{\partial \mathbf{U}_u^{(0)}}. \end{aligned} \quad (21)$$

The propagation process of GCN is repeated in the user layer and item layer alternatively, and the back propagation in the deeper layer is just the repeated application of the chain rule. Therefore, we only need to focus on the gradient computation of two adjacent layers, e.g., the l -th layer and $(l+1)$ -th layer. We decompose the derivation step by step through the chain rule, and finally obtain a formula that includes the indicator function $\mathbb{I}(\cdot)$ and the adaptive gradient scaling factor $\frac{1}{2|\mathcal{I}_u|}$. Here, we take the analysis of

the derivative of the $(l + 1)$ -th layer hash code of user u with respect to the l -th layer hash code of item i as an example, which is as follows:

$$\begin{aligned}
\frac{\partial \mathbf{h}_u^{(l+1)}}{\partial \mathbf{h}_i^{(l)}} &= \frac{\partial \text{sign}(\mathbf{s}_u^{(l)} \odot \mathbf{d}_u^{(l)})}{\partial \mathbf{h}_i^{(l)}} \\
&= \frac{\partial \text{sign}(\mathbf{s}_u^{(l)} \odot \mathbf{d}_u^{(l)})}{\partial (\mathbf{s}_u^{(l)} \odot \mathbf{d}_u^{(l)})} \odot \frac{\partial (\mathbf{s}_u^{(l)} \odot \mathbf{d}_u^{(l)})}{\partial \mathbf{h}_i^{(l)}} \\
&= \mathbf{1} \odot \left(\frac{\partial \mathbf{s}_u^{(l)}}{\partial \mathbf{h}_i^{(l)}} \odot \mathbf{d}_u^{(l)} + \mathbf{s}_u^{(l)} \odot \frac{\partial \mathbf{d}_u^{(l)}}{\partial \mathbf{h}_i^{(l)}} \right) \\
&= \frac{\partial \mathbf{s}_u^{(l)}}{\partial \mathbf{m}_u^{(l)}} \odot \frac{\partial \mathbf{m}_u^{(l)}}{\partial \mathbf{h}_i^{(l)}} \odot \mathbf{d}_u^{(l)} \\
&\quad + \mathbf{s}_u^{(l)} \odot \frac{\partial \mathbf{d}_u^{(l)}}{\partial \mathbf{p}_u^{(l)}} \odot \frac{\partial \mathbf{p}_u^{(l)}}{\partial \mathbf{m}_u^{(l)}} \odot \frac{\partial \mathbf{m}_u^{(l)}}{\partial \mathbf{h}_i^{(l)}} \\
&= \mathbb{I}(-1 \leq \mathbf{m}_u^{(l)} \leq 1) \odot \mathbf{d}_u^{(l)} + \mathbf{s}_u^{(l)} \odot \frac{1}{2|\mathcal{I}_u|}.
\end{aligned} \tag{22}$$

Eq.(22) directly reflects the gradient flow path of our graph convolution rule in backpropagation, which is the key to understanding the model optimization. In particular, we focus on the expression $(\mathbb{I}(-1 \leq \mathbf{m}_u^{(l)} \leq 1) \odot \mathbf{d}_u^{(l)} + \mathbf{s}_u^{(l)} \odot \frac{1}{2|\mathcal{I}_u|})$, which is because it shows the core advantage of our convolution rule. Specifically, it ensures the stability and effectiveness of gradient propagation through two key mechanisms. **Firstly**, the indicator function $\mathbb{I}(-1 \leq \mathbf{m}_u^{(l)} \leq 1)$ is a two-path conditional gradient flow mechanism, which can solve the problem of model training instability caused by unbalanced signal of the interactions of user u . When the signal is weak, i.e., $\mathbf{m}_u^{(l)} \in [-1, 1]$, both gradient paths are activated simultaneously. One is the derivative of $\mathbf{s}_u^{(l)}$, i.e., $\mathbb{I}(-1 \leq \mathbf{m}_u^{(l)} \leq 1) \odot \mathbf{d}_u^{(l)}$. At this point, this path is reflected as users with higher credibility weight have higher gradients. Another is the derivative of $\mathbf{d}_u^{(l)}$, i.e., $\mathbf{s}_u^{(l)} \odot \frac{1}{2|\mathcal{I}_u|}$, which provides stable gradients depending on the direction of the signal. When the signal is strong, i.e., $\mathbf{m}_u^{(l)} \in [-\infty, -1) \cup (1, +\infty]$, the first gradient path is closed. The second path only relies on $\mathbf{s}_u^{(l)} \odot \frac{1}{2|\mathcal{I}_u|}$, which avoids the gradient explosion problem caused by $\mathbf{m}_u^{(l)}$ and the problem of vanishing gradient caused by the disappearance of the first path, thus ensuring the stability of the model training process.

Secondly, $\frac{1}{2|\mathcal{I}_u|}$ introduces an adaptive gradient scaling mechanism based on user activity, which makes the value of the gradient inversely proportional to the number of items interacted by user u , which effectively balances the contributions of different users to the model update. For highly active users, a large value of $|\mathcal{I}_u|$ will automatically reduce the value of their gradient, preventing some active users from dominating the training process. However, for relatively inactive users, a small value of $|\mathcal{I}_u|$ will correspondingly increase their gradient influence, ensuring that their preferences can also be fully learned. This adaptive mechanism greatly improves the balance of model training.

From Eq.(22), we emphasize the importance of the design of $\mathbf{d}_u^{(l)}$. Specifically, if we do not introduce the additional transformation, i.e., $\mathbf{d}_u^{(l)} = |\mathbf{p}_u^{(l)}|$, when the value of the k -th bit of $\mathbf{p}_u^{(l)}$ is equal to 0, the k -th bit of $\mathbf{d}_u^{(l)}$ is also equal to 0. Moreover, since $\mathbf{p}_u^{(l)} = \frac{\mathbf{m}_u^{(l)}}{|\mathcal{I}_u|}$, we can obtain that the k -th bit of $\mathbf{m}_u^{(l)}$ is equal to 0. And since $\mathbf{s}_u^{(l)} = \text{clamp}(\mathbf{m}_u^{(l)})$, we can further see that the k -th bit of $\mathbf{s}_u^{(l)}$ is equal to 0. Therefore, when the value of the k -th bit of $\mathbf{p}_u^{(l)}$ is equal to 0, the k -th bit of $\frac{\partial \mathbf{h}_u^{(l+1)}}{\partial \mathbf{h}_u^{(l)}}$ is equal to 0, which causes the problem of vanishing gradient. Therefore, the additional transformation is necessary and helps us avoid the problem of vanishing gradient during backpropagation.

4.4. TriPro

In this section, we first analyze the objects of privacy protection, introduce our basic privacy-preserving framework and **make an attempt**, and then detail our TriPro and give the proof regarding information loss. Finally, we provide some analysis on complexity and privacy protection.

4.4.1. Protected Objects

As shown in Figure 1, our HashRec can be divided into two convolutional paths, which are learning user hash codes and item hash codes.

In the first path, each user keeps his or her own first-order subgraph locally, which contains the information about the items that are interacted with the corresponding user. Moreover, the hash codes of all items are publicly available in the server, which can be easily obtained by each user to learn his or her hash code. Hence, we consider that there is no risk of privacy leakage in the first path.

Next, we mainly analyze the second path and identify the following two data that may leak user privacy:

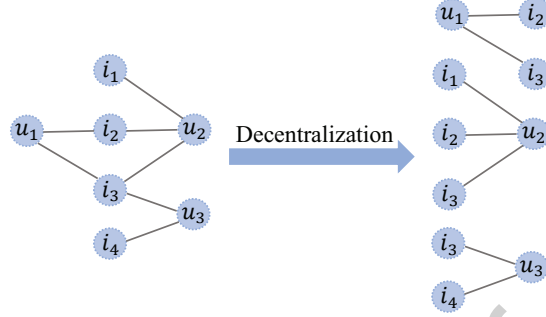


Figure 2: Illustration of decentralizing a global user-item bipartite graph.

- **User hash codes.** The server learns the hash code of an item by aggregating the hash codes of the neighbors. However, the hash code of each user is privacy sensitive and cannot be sent to the server, since otherwise the server can easily infer the user's preferences by combining the item hash codes.
- **User interaction records.** The server needs to know the positive user interaction records to ensure that the correct user hash codes are aggregated. Unfortunately, directly uploading these raw records exposes the users' historical records, from which malicious attackers can obtain more user privacy information.

Therefore, our goal is to design a privacy-preserving framework to transmit the privacy sensitive information as securely and completely as possible.

4.4.2. Basic Framework and Attempt

There are three main frameworks of federated learning, which are homomorphic encryption [45, 39], differential privacy [46, 47] and secret sharing [37, 48], respectively. Homomorphic encryption can provide strong security guarantees, but its computational overhead is huge. Differential privacy achieves privacy protection by introducing noise, but it will reduce the accuracy of the model again on the basis of learning to hash. Secret sharing splits a sensitive data into multiple meaningless secret segments, and reconstructs the data through the distribution and aggregation of those secret segments. Compared with the other two, secret sharing can reduce the computational overhead while ensuring that the accuracy of hash learning is not damaged. Based on the above analysis, we choose secret sharing as the basic framework.

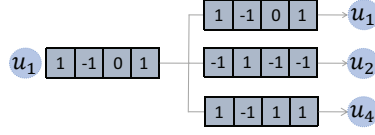


Figure 3: Illustration of splitting a user hash code into multiple segments and distributing them to different participants.

Here, we attempt to directly transfer protected objects via secret sharing. Before that, we design a fast splitting method to ensure that the secret segments are in the Hamming space. As shown in Figure 3, if user u_1 wants to send a secret (e.g., user hash code) to two participants, he or she will split the secret into three segments (one of which is kept locally). For each bit of these segments, the user randomly chooses some values in $\{-1, 0, 1\}$ by observing the sign of the corresponding bit in the hash code and makes sure that there is one more -1 than 1 . By doing this, we can quickly generate the secret segments.

For learning the hash code of each item i , in order to get the complete information of all neighbors of this item node, each user $u \in \mathcal{U}_i$ needs to perform secret sharing. In this case, although the secret sharing protects the hash codes of all users, their interactions are exposed to the corresponding participants who receive the secret segments. Therefore, some fake users $\mathcal{U}'_i$ also need to perform secret sharing to obfuscate the positive interaction of the users.

As shown in Figure 4, we show the update of the item hash code in a serial manner. Specifically, from the user's point of view, for each item i , each user $u \in \mathcal{U}_i \cup \mathcal{U}'_i$ must split his or her hash code into multiple secret segments and distribute them to different participants, while also aggregating the secret segments from other users if there is a reception. From the server's point of view, the server needs to continuously receive a huge number of secret segments from the users. As a result, this process introduces a serious communication and computation burden. Moreover, the server needs the number of the users who have interacted with the specific item, which requires performing additional secret sharing, again exacerbating the dilemma we face.

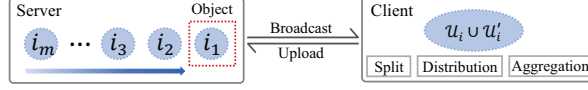


Figure 4: Illustration of secret sharing. When learning the hash code of the current object item $i = i_1$ (shown in a red dashed box), each user $u \in \mathcal{U}_i \cup \mathcal{U}'_i$ needs to perform split, distribution, aggregation (if received one or more secrets) of secrets.

4.4.3. Improved Framework

As we mentioned before, the reason of the severe communication and computational burden is the repeated invocation of secret sharing by users. It is easy to observe that when learning the $(l + 1)$ -th layer hash codes of all items, each user's hash code is fixed, so it is redundant to require users to perform secret sharing many times.

In order to solve this problem, we are inspired by the compatibility of computer systems and propose a secret sharing framework with triple protector named TriPro. The key idea is to make the secrets reusable. Specifically, as shown in Figure 5, we divide the secret into three function zones: adaptation zone, secret zone and verification zone, which are held by each user. To clarify the workflow of our method, we present the pseudocode of our FedHashRec in Algorithm 1, including the update process of the client and the server, as well as the communication between the client and the server (mainly shown in Algorithm 2).

In the following, we will detail how these three functional zones work from three aspects.

Division. Here, we introduce the information division strategy of these zones (line 2 in Algorithm 2).

- **The adaptation zone** contains multiple items, which are composed of all the positive items interacted by the corresponding user u and some negative items. Formally, it is denoted as $A_u = \mathcal{I}_u \cup \mathcal{I}'_u$. The negative items $\mathcal{I}'_u$ can mask the positive interaction items of the user. This mechanism ensures that neither the server nor other users can determine the positive interaction of the specific user, thus protecting the user's data privacy.
- **The secret zone** stores the segments of each user hash code. If user u intends to shares his or her secret with $P - 1$ users, then user u splits

Algorithm 1 The algorithm of our FedHashRec.

Initialization: Initialize model parameters $\mathbf{U}_u$ and $\mathbf{V}_i$, where $u \in \mathcal{U}$ and $i \in \mathcal{I}$.

```

1: for  $t = 1, \dots, T$  do
2:   Map the model parameters into the Hamming space via Eq.(6) and
   obtain the hash codes  $\mathbf{h}_u^{(0)}$  and  $\mathbf{h}_i^{(0)}$ .
3:   for each layer  $l = 1, \dots, L$  do
4:     for each client  $u \in \mathcal{U}$  do
5:       Client  $u$  randomly samples negative items  $\mathcal{I}'_u$ .
6:       Client  $u$  runs TriPro to update the fake secret  $F_u$  to the server.
7:       Client  $u$  downloads item hash codes  $\mathbf{h}_i^{(l-1)}$ ,  $i \in \mathcal{I}_u \cup \mathcal{I}'_u$ .
8:       Client  $u$  obtains the signal of neighbors by aggregating neighbor
       hash codes via Eq.(8).
9:       Client  $u$  clamps the signal via Eq.(11).
10:      Client  $u$  calculates the average signal strength via Eq.(9).
11:      Client  $u$  calculates the credibility weight via Eq.(10).
12:      Client  $u$  updates the  $l$ -th layer hash code  $\mathbf{h}_u^{(l)}$  via Eq.(12).
13:     end for
14:     The server extracts  $\mathbf{m}_i$  ( $i \in \mathcal{I}$ ) from the received secrets via Eq.(23).
15:     The server extracts  $|\mathcal{U}_i|$  ( $i \in \mathcal{I}$ ) from the received secrets via Eq.(24).
16:     The server performs the similar operations (lines 9-12) to the clients
       to update the  $l$ -th layer hash code  $\mathbf{h}_i^{(l)}$ .
17:   end for
18:   if model reaches the early stopping condition then
19:     Stop training.
20:   end if
21: end for

```

Algorithm 2 The algorithm of our TriPro.

Input: A set of users $\mathcal{U}$, the positive items $\mathcal{I}_u$, the negative items $\mathcal{I}'_u$ and the user hash codes $\mathbf{h}_u^{(l)}$, where $u \in \mathcal{U}$.

Output: A fake secret.

- 1: All clients perform the following operations in parallel.
 - 2: Client u locally divides his or her private sensitive information (i.e., $\mathcal{I}_u$ and $\mathbf{h}_u^{(l-1)}$) into the adaptation zone $A_u = \mathcal{I}_u \cup \mathcal{I}'_u$, secret zone $S_u = \{s_1, s_2, \dots, s_P\}$ and verification zone $V_u = \{v_{i1}, v_{i2}, \dots, v_{iQ}\}_{i \in A_u}$.
 - 3: Client u randomly combines A_u , S_u and V_u into multiple secrets.
 - 4: Client u keeps one of the secrets (denoted as $\{A_u, s_p^u, \{v_{iq}^u\}_{i \in A_u}\}$) and sends the rest to other clients.
 - 5: **if** client u receives some secrets **then**
 - 6: Client u merges all the adaption zones in these secrets, denoted as A'_u .
 - 7: Client u sums all the secret zones in these secrets, denoted as $s_p^{u'}$.
 - 8: Client u merges (not matched) or sums (matched) the verification segments in these secrets according to the adaption zones, denoted as $\{v_{iq}^{u'}\}_{i \in A'_u}$.
 - 9: Through the above three steps, client u obtains a fake secret, denoted as $F_u = \{A'_u, s_p^{u'}, \{v_{iq}^{u'}\}_{i \in A'_u}\}$.
 - 10: Client u uploads the fake secret F_u to the server.
 - 11: **end if**
 - 12: **if** client u does not receive any secret **then**
 - 13: The only secret he or she keeps is used as a fake secret ($F_u = \{A_u, s_p^u, \{v_{iq}^u\}_{i \in A_u}\}$) and uploaded to the server.
 - 14: **end if**
-

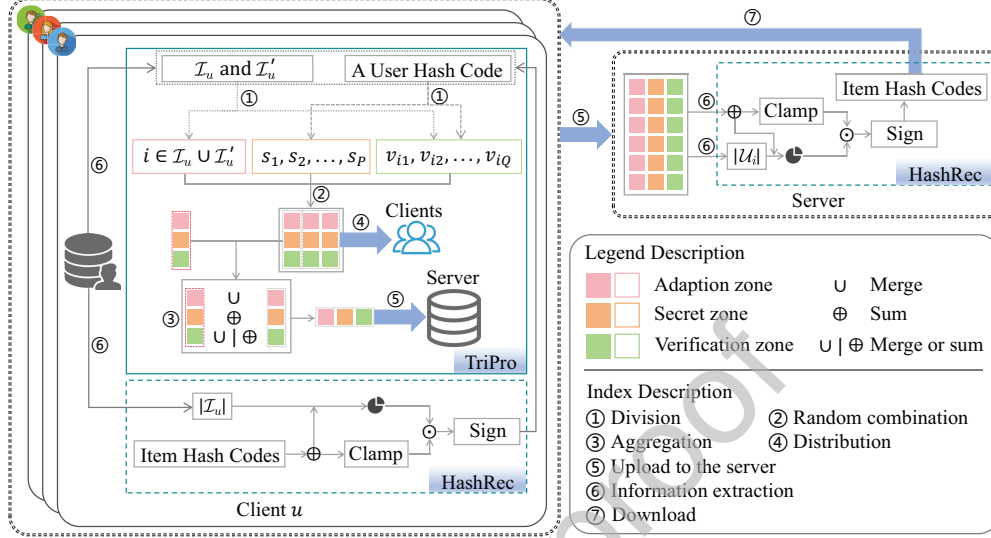


Figure 5: Illustration of our FedHashRec, which consists of HashRec and TriPro. For HashRec, the server and each user extract the information and update the associated hash codes (detailed in Section 4.1). For TriPro, each user divides his or her privacy sensitive information into three function zones (i.e., adaption zone, secret zone and verification zone), and randomly combines them into new secrets. Each user keeps one of the secrets and sends the rest to other participants. If a user receives one or more secrets from other participants (red dashed box), he or she needs to aggregate all the secrets, including his or her own. When all the users finish secret sharing, they upload their aggregated secrets to the server. The server generates a fake user-item bipartite graph based on the received secrets and uses it for training.

his or her own hash code into P secret segments $S_u = \{s_1, s_2, \dots, s_P\}$ by the fast splitting method mentioned in Section 4.4.2. The key of the information loss is the original user hash code can be reconstructed from the secret segments in S_u .

- **The verification zone** is composed of a series of verification segments of $K + 1$ dimensions, which is used to identify the positive and negative items in the adaption zone (the last dimension, i.e., the $(K + 1)$ -th dimension) and eliminate the noise caused by the negative items (the first K dimensions). Specifically, for each item $i \in A_u$, user u generates some verification segments, denoted as $V_u = \{v_{i1}, v_{i2}, \dots, v_{iQ}\}_{i \in A_u}$. We allow $P \neq Q$. For the sake of presentation, we denote the first K dimensions as V_u^{first} and the last dimension as V_u^{last} . If user u has

interacted with item i , the sum of V_u^{first} with respect to item i is a zero vector and the sum of V_u^{last} with respect to item i is 1. Otherwise, the sum of V_u^{first} with respect to item i is the negative of the hash code of user u and the sum of V_u^{last} with respect to item i is 0.

Combination and Distribution. When a user wants to share his or her hash code with others, he or she will first randomly combine the segments of the secret zone and the verification zone, and then pack each one together with the adaptation zone into multiple secrets (line 3 in Algorithm 2). Each user keeps one of these secrets and randomly sends the rest to other participants (line 4 in Algorithm 2). This strategy of randomly distributing meaningless secrets enhances the resistance to collusion attacks.

Local Aggregation and Upload. If a user receives some secrets, he or she first merges all the adaption zones and sums all the secret zones, and then merges (not matched) or sums (matched) the verification segments according to matching the items in his or her own adaptation zone (lines 6-8 in Algorithm 2). Thus, each user generates a fake secret (line 9 in Algorithm 2), which is public and uploaded to the server (line 10 in Algorithm 2). Especially, if a user does not receive any secret, he or she upload the kept secret as a fake secret (line 13 in Algorithm 2).

Our FedHashRec combines the graph convolution rule HashRec and the federated framework TriPro. In our FedHashRec, each user performs secret sharing only once in each convolution. When the server updates the hash codes of different items, these secrets uploaded by users are reused without additional secret sharing operations (lines 14-15 in Algorithm 1), which significantly reduces the communication and computation overhead by changing the granularity of secure aggregation from item-wise to layer-wise. Without TriPro, updating each item requires an independent secure-sharing process among all users in $\mathcal{U}_i \cup \mathcal{U}'_i$, which introduces repeated client-client communication and repeated local aggregation. Specifically, when using TriPro, the total communication complexity can be expressed as $\mathcal{O}(K \sum_u |\mathcal{I}_u \cup \mathcal{I}'_u|)$ if P and Q are constant. The total communication complexity is $\mathcal{O}(K \sum_i |\mathcal{U}_i \cup \mathcal{U}'_i|^2)$ when TriPro is not used. Therefore, TriPro reduces the dominant communication complexity from quadratic to linear with respect to the number of participating users per item (usually $|\mathcal{I}_u| < |\mathcal{U}_i|$).

Moreover, in the following, we prove that our FedHashRec has no information loss after applying the federated framework TriPro.

Proof. To prove the point, we just need to show that $\mathbf{m}_i^{(l)}$ and $|\mathcal{U}_i|$ calculated

in our HashRec can be obtained by our FedHashRec with no information loss. In this proof, we take the l -th layer graph convolution as an example. For convenience, we omit the superscript (l) of the variable in the following proofs.

According to the design of our FedHashRec, each user u divides the privacy sensitive information into three zones, which are adaptation zone A_u , secret zone S_u and verification zone V_u . Therein, $A_u = \mathcal{I}_u \cup \mathcal{I}'_u$ represents the set of the items interacted by user u and the negative items selected by user u , $S_u = \{s_1, s_2, \dots, s_P\}$ represents the set of the segments of user u 's hash code $\mathbf{h}_u$ which satisfies $\sum_{p=1}^P s_p^u = \mathbf{h}_u$ and $V_u = \{v_{i1}, v_{i2}, \dots, v_{iQ}\}_{i \in A_u}$ represents the set of the verification fragments for each item $i \in A_u$ which satisfies that $\sum_{q=1}^Q v_{iq}^u[1 : K] = \mathbf{0}$ and $\sum_{q=1}^Q v_{iq}^u[K] = 1$ if $i \in \mathcal{I}_u$, and $\sum_{q=1}^Q v_{iq}^u[1 : K] = -\mathbf{h}_u$ and $\sum_{q=1}^Q v_{iq}^u[K] = 0$ otherwise. Noted that we add the superscript u to mark that s^u and v^u belong to user u in this proof.

In secret sharing, user u randomly combines A_u , S_u , and V_u into multiple secrets, where the verification zone in a secret contains many verification segments from different $i \in A_u$. Then, he or she keeps one of them, denoted as $\{A_u, s_p^u, \{v_{iq}^u\}_{i \in A_u}\}$, and sends the rest to other participants. Let F_u denote the set of secrets retained and received by user u . In F_u , F_u^A denote the set of the adaptation zones, F_u^s denote the set of the secret segments and $F_u^{v,i}$ denote the set of the verification segments for item i . When the server learns the hash code of item i , we have:

$$\begin{aligned}
\mathbf{m}_i &= \sum_{u \in \mathcal{U}} \sum_{i \in F_u^A} \left(\sum_{f_u^s \in F_u^s} f_u^s + \sum_{f_u^{v,i} \in F_u^{v,i}} f_u^{v,i} [1 : K] \right) \\
&= \sum_{u \in \mathcal{U}} \sum_{i \in A_u} \left(\sum_{p=1}^P s_p^u + \sum_{q=1}^Q v_{iq}^u [1 : K] \right) \\
&= \sum_{u \in \mathcal{U}} \sum_{i \in \mathcal{I}_u \cup \mathcal{I}'_u} \left(\mathbf{h}_u + \sum_{q=1}^Q v_{iq}^u [1 : K] \right) \\
&= \sum_{u \in \mathcal{U}} \left(\sum_{i \in \mathcal{I}_u} \left(\mathbf{h}_u + \sum_{q=1}^Q v_{iq}^u [1 : K] \right) + \sum_{i \in \mathcal{I}'_u} \left(\mathbf{h}_u + \sum_{q=1}^Q v_{iq}^u [1 : K] \right) \right) \quad (23) \\
&= \sum_{u \in \mathcal{U}} \left(\sum_{i \in \mathcal{I}_u} (\mathbf{h}_u + \mathbf{0}) + \sum_{i \in \mathcal{I}'_u} (\mathbf{h}_u - \mathbf{h}_u) \right) \\
&= \sum_{u \in \mathcal{U}} \sum_{i \in \mathcal{I}_u} \mathbf{h}_u \\
&= \sum_{u \in \mathcal{U}_i} \mathbf{h}_u.
\end{aligned}$$

By Eq.(23), we find that $\mathbf{m}_i^{(l)}$ calculated in the centralized version of HashRec can be obtained by our FedHashRec with no information loss. Moreover, we prove that $|\mathcal{U}_i|$ also can be obtained with no information loss, which is as follows:

$$\begin{aligned}
|\mathcal{U}_i| &= \sum_{u \in \mathcal{U}} \sum_{i \in F_u^A} \sum_{f_u^{v,i} \in F_u^{v,i}} f_u^{v,i}[K] \\
&= \sum_{u \in \mathcal{U}} \sum_{i \in A_u} \sum_{q=1}^Q v_{iq}^u[K] \\
&= \sum_{u \in \mathcal{U}} \sum_{i \in \mathcal{I}_u \cup \mathcal{I}'_u} \sum_{q=1}^Q v_{iq}^u[K] \\
&= \sum_{u \in \mathcal{U}} \left(\sum_{i \in \mathcal{I}_u} \sum_{q=1}^Q v_{iq}^u[K] + \sum_{i \in \mathcal{I}'_u} \sum_{q=1}^Q v_{iq}^u[K] \right) \\
&= \sum_{u \in \mathcal{U}} \left(\sum_{i \in \mathcal{I}_u} 1 + \sum_{i \in \mathcal{I}'_u} 0 \right) \\
&= \sum_{u \in \mathcal{U}} \sum_{i \in \mathcal{I}_u} 1 \\
&= |\mathcal{U}_i|.
\end{aligned} \tag{24}$$

According to the above proof, we can make a conclusion that the secret sharing framework with triple protector we designed, i.e., TriPro, has no information loss after the data reaches the server. $\square$

4.5. Complexity and Privacy Analysis

In this section, we first analyze the communication complexity in our FedHashRec, and then analyze the privacy protection.

4.5.1. Complexity Analysis

The communication complexity mainly consists of two phases: secret sending (sending to other participants and the server) and receiving (receiving secrets from other participants and the item hash codes from the server). In the first phase, each user u holds P secrets, of which $P - 1$ are sent to participants. And after the secret sharing, a secret is sent to the server. Therefore, the communication complexity of sending this secret is the complexity of the secrets held by each user. The complexity of each user u

holds three parts, which account for the communication complexity of the adaptation zone $P \times |\mathcal{I}_u \cup \mathcal{I}'_u|$, the secret zone $P \times K$ and the verification zone $Q \times |\mathcal{I}_u \cup \mathcal{I}'_u| \times (K + 1)$ respectively. In the second phase, the average complexity of a secret (assume it is from user v) received by user u is $|\mathcal{I}_v \cup \mathcal{I}'_v| + K + \frac{Q}{P} |\mathcal{I}_v \cup \mathcal{I}'_v| (K + 1)$. Then the expected complexity of user u receiving secrets of other users is $\frac{P-1}{N-1} \sum_{v \neq u} (|\mathcal{I}_v \cup \mathcal{I}'_v| + K + \frac{Q}{P} |\mathcal{I}_v \cup \mathcal{I}'_v| (K + 1))$. Moreover, we can know that the communication complexity of each user u receiving the item hash codes from the server is $K \times |\mathcal{I}_u \cup \mathcal{I}'_u|$. To summarize, in each convolution, the communication complexity of each user u is $|\mathcal{I}_u \cup \mathcal{I}'_u| (P + Q(K + 1) + K) + PK + (P-1) (|\mathcal{I}_u \cup \mathcal{I}'_u| + K + \frac{Q}{P} |\mathcal{I}_u \cup \mathcal{I}'_u| (K + 1))$ (assume $|\mathcal{I}_u \cup \mathcal{I}'_u|$ is approximately the same for all users). Since $|\mathcal{I}_u \cup \mathcal{I}'_u| = (1 + \rho) \times |\mathcal{I}_u|$, we can find that when P , Q and K are constant, the communication complexity is proportional to the number of interacted items (i.e., $|\mathcal{I}_u|$).

4.5.2. Privacy Analysis

In our FedHashRec, the attackers are the clients and the server, which will try to attack by analyzing the intermediate data they receive. The goal of privacy protection is to make user interactions and hash codes not easily inferred.

Our FedHashRec sets up triple protection of privacy protection mechanisms at the adaptation zone, the secret zone and the verification zone, which we analyze in the following.

The adaptation zone is used to obfuscate the historical interaction and thus protect user interactions. For other clients, they receive the adaptation zone of user u , i.e., $A_u = \mathcal{I}_u \cup \mathcal{I}'_u$. The probability that another client w guesses that an item id is positive is $\frac{|\mathcal{I}_u|}{|\mathcal{I}_u \cup \mathcal{I}'_u|}$ and the probability that he or she correctly guesses the attribute of all items (positive or negative) is $\frac{1}{2^{|\mathcal{I}_u \cup \mathcal{I}'_u|}}$. For the server, it receives the adaptation zone of user u which is merged with other adaptation zones that user u receives from other clients, denoted as A'_u . It's obvious that $|A'_u| \geq |A_u|$, and hence it is more difficult for both the server and a malicious attacker to infer the user interactions.

The secret zone is used to hide the real hash code of each user u . For other clients, user u splits his or her hash code into P segments and sends $P - 1$ segments to other clients. Since user u keeps one of the segments, other clients collude to leak single or multiple segments without revealing the hash code of user u . From the point of view of information theory, the attacker

who holds some segments gets the same posterior probability about the hash code as the prior probability, that is, these segments do not provide any information about the hash code. We can further mathematically describe it as $H(S|S_{\text{leakage}}) = H(S)$, where $H(\cdot|\cdot)$ denotes the conditional entropy, S denotes the hash code of user u and S_{leakage} denotes the set of leaked segments, which means that even if some segments are leaked, the malicious attackers still cannot determine what is the correct hash code. For the server, the secret zone uploaded by user u is aggregated with secret segments received from other clients, which leaks no information, so the same conclusion can be obtained as above. The probability that the other clients and the server infer the hash code of user u is $\frac{1}{3K}$, which is completely random.

The verification zone has two roles, which are to eliminate the noise caused by the negative items (the first K dimensions) and to identify the positive and negative items (the $(K + 1)$ -th dimension). The verification segments generated by user u for each $i \in A_u$ is meaningless and reveal no information about the interactions, the number of the interactions, and the hash code of user u . Therefore, consistent with the privacy analysis of the adaptation zone and secret zone, the verification segments do not leak the interactions, the positive interactions and hash code of user u , whether held by other clients or by the server.

5. Experiments

In this section, we conduct experiments on four datasets and answer the following research questions:

- **RQ1.** How dose our FedHashRec perform compared with [the representative recommendation models](#)?
- **RQ2.** What is the performance difference between our FedHashRec and HashRec?
- **RQ3.** What is the resource consumption of our HashRec?
- **RQ4.** How do the dimension size K and the weight parameters λ_1 and λ_2 affect the performance of our HashRec?
- **RQ5.** What is the quality of the user and item hash codes learned by our HashRec?

- **RQ6.** What are the advantages of the model components in our HashRec?

5.1. Datasets

We use four real-world datasets in our experiments, i.e., MovieLens 1M (ML1M)², Netflix³, XING⁴ and Amazon-KindleStore (Amazon-KS)⁵, which are widely used in Top-K recommendation tasks. For both Netflix and XING, we randomly sample 5000 users, and 5000 items, and then generate two new datasets, namely Netflix5K5K (NF5K5K) and XING5K5K. For Amazon-KS, we keep records where there are at least 10 interactions between users and items. After that, we further preprocess these datasets as follows [49]:

1. For the rating dataset, we keep only the records with ratings greater than 3 as observed positive records.
2. We randomly sample 50% observed records as a training set, and the rest 50% as a test set.
3. We randomly take one user-item pair for each user from the training set to construct a validation set.
4. We repeat the second step and the third step 3 times to get three different copies, where each copy contains a training set, a validation set and a test set.

Note that the experimental results we report are the average performance of three copies. The statistics of the processed datasets are summarized in Table 2.

Table 2: Statistics of the processed datasets.

Dataset	ML1M	NF5K5K	XING5K5K	Amazon-KS
#Users	6040	5000	5000	9862
#Items	3952	4998	5000	11298
#User-item pairs	575,281	155,872	78,294	245,458
Density	2.41%	0.62%	0.31%	0.22%

²<https://grouplens.org/datasets/movielens/>.

³<http://www.netflixprize.com/>.

⁴<https://www.xing.com/>.

⁵<https://jmcauley.ucsd.edu/data/amazon/>.

5.2. Evaluation Metrics

In a recommendation task, since users pay more attention to the top items in the recommendation list, we choose three widely recognized evaluation metrics for quantitative analysis, i.e., normalized discounted cumulative gain (NDCG@), precision (Pre@) and recall (Rec@).

5.3. Baselines

In the experiments, we compare our FedHashRec with the following representative hashing-based recommendation models.

- LSH [16] is a classical hashing method, which is used for approximate nearest neighbor search in a high dimensional space. The basic idea is that the hash function maps similar points to a same bucket with high probability. We apply LSH and use cosine similarity to recommend items for users. It should be noted that LSH for recommendation is a similarity-based method, which is not a discrete method. The core of LSH is search efficiency, so in our experiments it is used as a strong baseline in the time consumption study.
- LightGCN [5] is a simple GCN model, which learns user and item embeddings by linearly propagating them on the user-item bipartite graph.
- FedPerGNN [11] is a federated GNN model, which introduces a trusted third-party server to build similar user neighborhoods for each client to learn high-order information.
- P-GCN [50] is a federated recommendation model, which employs secure aggregation and graph-wise concealing strategies to build a privacy-enhanced user-item graph.
- FedHoG [51] is a federated homogeneous graph construction method, which models users and items by constructing an item-item graph and a user-user graph.
- DCF [17] is a collaborative filtering method based on discrete representation, which is designed for the recommendation problem based on explicit feedback. It discretizes the embedding vectors of users and

items into binary codes, and maintains the similarity relationship between users and items by optimizing the binary codes, so as to achieve efficient recommendation.

- HashGNN_s is a GCN-based hashing method for top-N candidate generation for recommendation. HashGNN_s denotes the approximated version proposed in HashGNN [22]. In this version, the elements of the user and item embeddings contain the [full-precision](#) and binarized values.
- HashGNN_h is denoted as a strictly binarized version of HashGNN [22]. Different from HashGNN_s, instead of using Bernoulli random variables to select from continuous embeddings and hash codes, we keep only the hash codes. It is a [fully discrete](#) model based on GCN, which is the same as our setting.
- HS-GCN [21] is a GCN-based hashing method. HS-GCN makes the embeddings of users and items gradually tend to be binary by using the proxy function, and finally generates a binary hash code. It's the one most relevant model to ours.

5.4. Parameter Settings

For fairness, we set the following parameters of all baselines to be same: the maximum number of the iterations $T = 500$, the random seed $s = 0$, the dimension of the latent feature vector $K = 128$ ([the default configuration is \$K = 8\$ for the full-precision models](#)), the user batch size $\tau = 128$, the negative sampling rate $\rho = 1$ and the number of the iterations for the early stopping strategy $T_{\text{stop}} = 50$. In addition, for each baseline, we search the best values of other parameters according to NDCG@50 on the validation set of the first copy of each dataset. For our FedHashRec, we use the optimal parameters on our HashRec. Specifically, on ML1M, we perform a grid search over the following ranges: the trade-off parameter of the ranking loss $\lambda_1 \in \{1e-6, 1e-5, 1e-4, 1e-3, 1e-2, 1e-1\}$, the trade-off parameter of the sign constrained loss $\lambda_2 \in \{1e-9, 1e-8, 1e-7, 1e-6, 1e-5, 1e-4\}$ and the learning rate $\gamma \in \{1e-3, 1e-2, 1e-1, 1\}$. The optimal configurations are determined to be $\lambda_1 = 1e-2, \lambda_2 = 1e-7, \gamma = 1e-2$. On NF5K5K, the search spaces are defined as $\lambda_1 \in \{1e-7, 1e-6, 1e-5, 1e-4, 1e-3, 1e-2\}$, $\lambda_2 \in \{1e-7, 1e-6, 1e-5, 1e-4, 1e-3, 1e-2, 1e-1\}$ and $\gamma \in \{1e-3, 1e-2, 1e-1, 1\}$. The optimal parameter values on this dataset are $\lambda_1 = 1e-6, \lambda_2 = 1e-5, \gamma = 1e-1$. [On](#)

Amazon-KS, the search spaces are defined as $\lambda_1 \in \{1e-6, 1e-5, 1e-4, 1e-3, 1e-2\}$, $\lambda_2 \in \{1e-8, 1e-7, 1e-6, 1e-5, 1e-4, 1e-3, 1e-2, 1e-1\}$ and $\gamma \in \{1e-4, 1e-3, 1e-2, 1e-1, 1\}$. The optimal parameter values on this dataset are $\lambda_1 = 1e-5, \lambda_2 = 1e-7, \gamma = 1e-1$. On **XING5K5K**, the search spaces are defined as $\lambda_1 \in \{1e-6, 1e-5, 1e-4, 1e-3, 1e-2\}$, $\lambda_2 \in \{1e-7, 1e-6, 1e-5, 1e-4, 1e-3, 1e-2, 1e-1\}$ and $\gamma \in \{1e-3, 1e-2, 1e-1, 0.5, 1, 1.5, 2\}$. The optimal parameter values on this dataset are $\lambda_1 = 1e-3, \lambda_2 = 1e-6, \gamma = 1$.

We reproduce LSH, DCF, **HS-GCN** and **LightGCN** algorithms in Python according to the details of the corresponding papers. For HashGNN, **FedPerGNN** and **P-GCN**, we use the source code provided by the authors. For the non-federated baselines, we perform a grid search to find the optimal parameters. Since a federated recommendation model requires repeated local training and server aggregation, the hyperparameter tuning for each baseline is computationally expensive. Therefore, for graph-based recommendation baselines such as P-GCN, FedPerGNN and FedHoG, we adopt the hyperparameter settings of LightGCN (including the regularization parameter and learning rate), which is a representative and widely used graph collaborative filtering model.

5.5. Experimental Results

5.5.1. Performance Comparison (RQ1)

In this section, we evaluate the representation ability of the hash codes generated by our FedHashRec on four datasets. Specifically, we obtain the top- K recommended items by measuring the distance between the hash code of each user and that of each candidate item and sorting them. Table 3 reports the results on NDCG, Pre and Rec. From the table, we have the following **seven** observations. (i) LSH achieves good recommendation performance on all datasets. This is due to that the key of LSH for recommendation is to mine recommended items from searched similar users according to association rules. This traditional method is still competitive in recommender systems, especially on XING5K5K. However, LSH has insufficient generalization ability when recommending more items. For example, on NF5K5K, NDCG@50 decreases by 41.1% compared with NDCG@5. This is because as the length of the recommendation list increases, items with low similarity are recommended, leading to a decrease in accuracy. It should be noted that since LSH is essentially a **full-precision** algorithm, we take it as a reference in this research question. Notice that the performance of our FedHashRec

is better than LSH in most cases, indicating that our method can match or even outperform the performance of some traditional **full-precision** models. (ii) DCF performs significantly worse than other baselines on the datasets with dense interactions such as ML1M and NF5K5K, but has relatively better performance on XING5K5K and Amazon-KS. This is because DCF is a hashing method based on collaborative filtering with a simple model structure. On these **relatively** dense datasets, DCF is difficult to capture complex user-item relationships, resulting in significantly lower model performance than other baselines. On the contrary, simple collaborative filtering has better performance on sparse datasets. (iii) The model performance of HashGNN_s is consistently better than that of HashGNN_h on all datasets. This is due to the fact that strict binarization leads to more information loss, resulting in poor representation ability of the embeddings and lower model performance. (iv) GCN-based hashing methods, such as HashGNN_s, HashGNN_h and HS-GCN, have poor performance on XING5K5K and Amazon-KS. This is due to the lack of connections between nodes in these relatively sparse datasets, making it difficult to aggregate high-quality neighborhood information, so that the performance of these models cannot be improved. (v) On NF5K5K, our FedHashRec outperforms the most hashing methods and is slightly lower than HashGNN_s in some metrics. In Table 4, we can find that the model performance of our HashRec is significantly better than that of HashGNN_s on NF5K5K. From this, we can conclude that the lower performance of FedHashRec than HashGNN_s is caused by the training mechanism of federated learning. We summarize two possible reasons for this. One is that on NF5K5K, the behavior of each user may be relatively different, resulting in significant deviations between the local data distribution and the global data distribution. Another one is that our FedHashRec and HashGNN_s are optimized in different ways. The latter uses batch gradient descent, which has a smooth optimization path, while the former uses the federated average algorithm, which averages the updates and then optimizes, which introduces some noise. However, compared to the strictly binarized version HashGNN_h, our FedHashRec is superior in performance. (vi) On ML1M, XING5K5K and Amazon-KS, our FedHashRec comprehensively outperforms other hashing methods. This is due to the fact that the training mechanism we designed can improve the performance of the model while keeping the model discretization. Especially on the sparse datasets, our FedHashRec has achieved very promising results (e.g., most of the metrics are increased by more than 100% compared with HS-GCN), which shows that our method

has the ability to effectively preserve graph collaborative signals under discrete representation. (vii) For comparing with full-precision models, our FedHashRec is compared with LightGCN, FedPerGNN, P-GCN and FedHoG, which have the same embedding storage cost. Our FedHashRec consistently outperforms LightGCN on ML1M, XING5K5K and Amazon-KS, which is because LightGCN captures high-order information by simple neighborhood aggregation on the user-item bipartite graph, and its performance is limited by the embedding dimension. In LightGCN, low-dimensional embeddings are not sufficient to learn user preferences. P-GCN improves on LightGCN and adopts item-based user representation to compensate for the loss it imposes on modeling the high-order connectivity. On most datasets, our FedHashRec is comparable to P-GCN. In particular, on ML1M and XING5K5K, our FedHashRec performs better. FedPerGNN performs better than our FedHashRec on ML1M and NF5K5K, and worse on XING5K5K and Amazon-KS. This is because that FedPerGNN introduces a graph expansion protocol, which contains more information in the extended graph when a dataset is relatively dense. FedHoG outperforms our FedHashRec on all datasets due to the fact that FedHoG introduces a more complex mechanism, i.e., modeling high-order collaborative relations by constructing homogeneous graphs (i.e., user-user and item-item graphs). However, in the training process, FedHoG frequently uses homomorphic encryption and decryption operations, which significantly increases the computational overhead. In addition, the convolution process of users and items is performed entirely in the server, which may expose user preferences, causing privacy leakage risk. It should be noted that the goal of our FedHashRec is not to dominate all full-precision graph-based methods in terms of recommendation accuracy. The full-precision models preserve more fine-grained continuous preference information, so it is reasonable that these methods exhibit better performance on some datasets. The key advantage of our FedHashRec is under the same storage budget, which effectively compensates for the precision loss caused by discretization and preserves useful collaborative signals, achieving competitive recommendation performance using ternary embeddings.

5.5.2. Performance Difference (RQ2)

In this experiment, we evaluate the performance difference between our FedHashRec and HashRec, that is, the impact of the privacy-preserving framework TriPro on the performance of our HashRec. We report the experimental results in Table 4 and have the following observations. (i) The model

Table 3: Recommendation performance comparison on four datasets. The results of LSH is included for reference. The best and second-best results of the discrete methods are marked in bold and underlined, respectively. % Improv. denote the relative improvements (%) compared with HS-GCN.

Dataset	Method	NDCG@5	NDCG@20	NDCG@50	Pre@5	Pre@20	Pre@50	Rec@5	Rec@20	Rec@50
ML1M	LSH	0.2691	0.2117	0.1956	0.2546	0.1764	0.1292	0.0334	0.0863	0.1507
	<u>LightGCN</u>	0.2881	0.2558	0.2538	0.2766	0.2216	0.1687	0.0406	0.1230	0.2220
	<u>FedPerGNN</u>	0.4028	0.3539	0.3557	0.3903	0.3037	0.2321	0.0632	0.1787	0.3168
	<u>P-GCN</u>	0.3164	0.2789	0.2764	0.3083	0.2420	0.1854	0.0447	0.1327	0.2397
	<u>FedHoG</u>	0.4420	0.3822	0.3815	0.4268	0.3247	0.2461	0.0701	0.1934	0.3366
	DCF	0.0612	0.0647	0.0723	0.0620	0.0649	0.0658	0.0046	0.0202	0.0534
	HashGNN _s	0.2225	0.2131	<u>0.2269</u>	0.2171	<u>0.1891</u>	<u>0.1541</u>	0.0343	<u>0.1137</u>	<u>0.2155</u>
	HashGNN _h	0.1831	0.1760	0.1881	0.1806	0.1584	0.1308	0.0272	0.0928	0.1795
	HS-GCN	<u>0.2376</u>	<u>0.2134</u>	0.2179	<u>0.2336</u>	0.1869	<u>0.1457</u>	<u>0.0354</u>	0.1080	0.2014
	FedHashRec	0.3287	0.2830	0.2847	0.3173	0.2431	0.1887	0.0475	0.1376	0.2534
% Improv.		38.32%	32.59%	30.65%	35.86%	30.09%	29.57%	34.37%	27.39%	25.85%
NF5K5K	LSH	0.1557	0.1133	0.0917	0.1404	0.0797	<u>0.0399</u>	0.0340	0.0590	0.0652
	<u>LightGCN</u>	0.1682	0.1787	0.2155	0.1592	0.1274	<u>0.0963</u>	0.0513	0.1773	0.3143
	<u>FedPerGNN</u>	0.1294	0.1685	0.2126	0.1255	0.1210	0.0936	0.0551	0.1829	0.3310
	<u>P-GCN</u>	0.1784	0.1792	0.2157	0.1718	0.1299	0.0982	0.0504	0.1676	0.3067
	<u>FedHoG</u>	0.1524	0.1585	0.1945	0.1448	0.1224	0.0939	0.0387	0.1390	0.2842
	DCF	0.0473	0.0505	0.0670	0.0478	0.0469	0.0458	0.0081	0.0337	0.0852
	HashGNN _s	<u>0.1443</u>	0.1634	0.2064	<u>0.1373</u>	0.1165	0.0921	0.0528	0.1652	0.3065
	HashGNN _h	0.1000	0.1147	0.1527	0.0963	0.0850	0.0714	0.0338	0.1148	0.2351
	HS-GCN	0.1351	0.1347	0.1664	0.1269	0.0989	0.0791	0.0391	0.1201	0.2347
	FedHashRec	0.1487	0.1518	0.1815	0.1384	0.1067	0.0825	0.0502	0.1395	0.2470
% Improv.		10.01%	12.70%	9.11%	9.01%	7.82%	4.30%	28.45%	16.15%	5.27%
XING5K5K	LSH	0.1073	0.1192	0.1439	0.1073	0.1088	0.0988	0.0213	0.0786	0.1837
	<u>LightGCN</u>	0.0426	0.0433	0.0495	0.0395	0.0327	0.0281	0.0118	0.0342	0.0647
	<u>FedPerGNN</u>	0.0446	0.0466	0.0565	0.0425	0.0354	0.0307	0.0132	0.0402	0.0794
	<u>P-GCN</u>	0.0402	0.0431	0.0487	0.0388	0.0353	0.0304	0.0092	0.0331	0.0632
	<u>FedHoG</u>	0.0591	0.0667	0.0861	0.0553	0.0456	0.0392	0.0223	0.0682	0.1352
	DCF	0.0361	0.0373	0.0452	<u>0.0351</u>	<u>0.0322</u>	<u>0.0293</u>	0.0074	0.0269	0.0596
	HashGNN _s	0.0351	<u>0.0438</u>	<u>0.0617</u>	0.0339	0.0314	0.0283	0.0129	<u>0.0480</u>	0.1033
	HashGNN _h	0.0348	0.0423	0.0584	0.0331	0.0300	0.0266	<u>0.0131</u>	0.0454	0.0960
	HS-GCN	0.0215	0.0231	0.0268	0.0213	0.0212	0.0210	0.0037	0.0136	0.0337
	FedHashRec	0.0467	0.0531	0.0658	0.0435	0.0383	0.0332	0.0166	0.0500	<u>0.0963</u>
% Improv.		116.64%	129.80%	146.01%	103.78%	80.57%	57.79%	346.85%	268.45%	185.90%
Amazon-KS	LSH	0.0499	0.0394	0.0383	0.0451	0.0199	0.0093	0.0200	0.0313	0.0358
	<u>LightGCN</u>	0.0326	0.0374	0.0498	0.0308	0.0225	0.0169	0.0148	0.0411	0.0738
	<u>FedPerGNN</u>	0.0473	0.0571	0.0775	0.0452	0.0351	0.0268	0.0221	0.0648	0.1187
	<u>P-GCN</u>	0.0922	0.1058	0.1371	0.0859	0.0616	0.0443	0.0430	0.1160	0.2004
	<u>FedHoG</u>	0.0861	0.0992	0.1272	0.0815	0.0584	0.0416	0.0407	0.1095	0.1851
	DCF	0.0755	0.0851	0.1054	0.0704	0.0486	0.0333	0.0367	<u>0.0921</u>	0.1465
	HashGNN _s	0.0383	0.0486	0.0704	0.0371	0.0313	0.0258	0.0174	0.0566	0.1129
	HashGNN _h	0.0353	0.0463	0.0678	0.0342	0.0297	0.0250	0.0159	0.0546	0.1099
	HS-GCN	0.0389	0.0448	0.0554	0.0363	0.0263	0.0179	0.0181	0.0491	0.0778
	FedHashRec	0.0806	0.0885	0.1080	0.0746	0.0495	<u>0.0331</u>	0.0387	0.0943	0.1474
% Improv.		107.38%	97.75%	94.83%	105.44%	88.59%	85.15%	114.16%	91.98%	89.40%

performance of our FedHashRec outperforms HashRec on ML1M. This shows that applying the proposed framework TriPro, while preserving privacy, not only does not reduce the performance of the model, but also brings performance improvement through the distributed training mechanism. We think that our TriPro achieves no information loss during training, which lays the foundation for model performance improvement. (ii) On Amazon-KS, our

FedHashRec slightly outperforms our HashRec on most metrics and only slightly underperforms on Pre@50 and Rec@50. This may be due to the fact that hot items get higher exposure when aggregating gradients, leading to more accurate recommendations at the top. (iii) There is a slight degradation in the model performance of our FedHashRec on NF5K5K and XING5K5K. Although there is no information loss with applying our designed privacy-preserving framework, the server aggregates the gradients from the clients, which introduces some noise. Therefore, it is acceptable in federated learning that the noise has a slight negative impact on the aggregated global model. In summary, our FedHashRec achieves comparable or better performance than our HashRec on different datasets, proving that our designed privacy-preserving framework is effective.

Table 4: Performance difference between our FedHashRec and HashRec

Dataset	Method	NDCG@5	NDCG@20	NDCG@50	Pre@5	Pre@20	Pre@50	Rec@5	Rec@20	Rec@50
ML1M	HashRec	0.3089	0.2661	0.2635	0.2977	0.2307	0.1778	0.0432	0.1239	0.2252
	FedHashRec	0.3287	0.2830	0.2847	0.3173	0.2431	0.1887	0.0475	0.1376	0.2534
NF5K5K	HashRec	0.1537	0.1573	0.1877	0.1440	0.1113	0.0853	0.0519	0.1457	0.2548
	FedHashRec	0.1487	0.1518	0.1815	0.1384	0.1067	0.0825	0.0502	0.1395	0.2470
XING5K5K	HashRec	0.0472	0.0533	0.0669	0.0445	0.0386	0.0337	0.0175	0.0500	0.0972
	FedHashRec	0.0467	0.0531	0.0658	0.0435	0.0383	0.0332	0.0166	0.0500	0.0963
Amazon-KS	HashRec	0.0778	0.0862	0.1072	0.0727	0.0494	0.0341	0.0370	0.0925	0.1491
	FedHashRec	0.0806	0.0885	0.1080	0.0746	0.0495	0.0331	0.0387	0.0943	0.1474

5.5.3. Resource Consumption (RQ3)

Since all baselines are centralized, our HashRec (the centralized version) is used to compare with them to avoid the impact of federated training on resource consumption. We analyze the time consumption of training and the storage consumption of the embedding as follows.

- Time consumption. As shown in Figure 6, we have the following observations. (i) LSH and DCF are strongly correlated with the size of a dataset. On ML1M, NF5K5K and XING5K5K, they have comparable time consumption since the sizes are comparable. On Amazon-KS, higher time consumption is caused by the large dataset. The difference is that LSH spends more time on searching for similarity users and recommending items, while DCF spends more time on learning embeddings. (ii) HashGNN_s and HashGNN_h have high time consumption on all datasets, which is due to its high computational complexity. In addition, we observe that HashGNN_h is usually more time-consuming than HashGNN_s due to the strict binarization, which takes more time

to converge. (iii) HS-GCN and our HashRec show high efficiency on all datasets because they use a more efficient graph convolution strategy. However, we observe that the time consumption of HS-GCN skyrockets on Amazon-KS. This is because Amazon-KS is a large and sparse dataset, which makes it difficult for a GCN to obtain effective information during the process of [neighborhood aggregation](#), and the gradient is unstable, which makes it difficult to meet the early stopping condition. In contrast, our HashRec achieves fast convergence by improving the convolution rule and the optimization objective. On all datasets, our HashRec achieves the highest training efficiency while maintaining favourable recommendation performance.

- **Storage consumption.** We report in Table 5 the storage consumption for the embeddings of different methods (except LSH, which is a non-embedding-based method). We can see that DCF, HashGNN_h and our HashRec require the same storage for embeddings in the training phase and the inference phase, this is because they are all [fully discrete](#) methods. Differently, the embeddings of our HashRec is in the ternary Hamming space and requires two bits for each value. HashGNN_s is an incomplete discrete method that includes both binary and continuous values in its embeddings. As a result, the storage consumption of its embeddings is uncertain and intermediate between approaches with [fully discrete](#) and [full-precision](#) methods. HS-GCN adopts different strategies in the training phase ([full-precision](#)) and the inference phase ([fully discrete](#)), so it has different storage consumption.

In conclusion, our HashRec can achieve promising recommendation performance with minimal time and relatively low storage consumption, which indirectly indicates the potential of our FedHashRec for efficient deployment in distributed environments.

Table 5: Storage consumption (bits) comparison on different methods.

Method	DCF	HashGNN _s	HashGNN _h	HS-GCN	HashRec
Training	$K \times (m+n)$	$[K \times (m+n), 32 \times K \times (m+n)]$	$K \times (m+n)$	$32 \times K \times (m+n)$	$2 \times K \times (m+n)$
Inference	$K \times (m+n)$	$[K \times (m+n), 32 \times K \times (m+n)]$	$K \times (m+n)$	$K \times (m+n)$	$2 \times K \times (m+n)$

5.5.4. Hyperparameter Sensitivity Analysis (RQ4)

In this section, we study the impact of three hyperparameters, i.e., the dimension size K and the two weight parameters λ_1 and λ_2 , on the perfor-

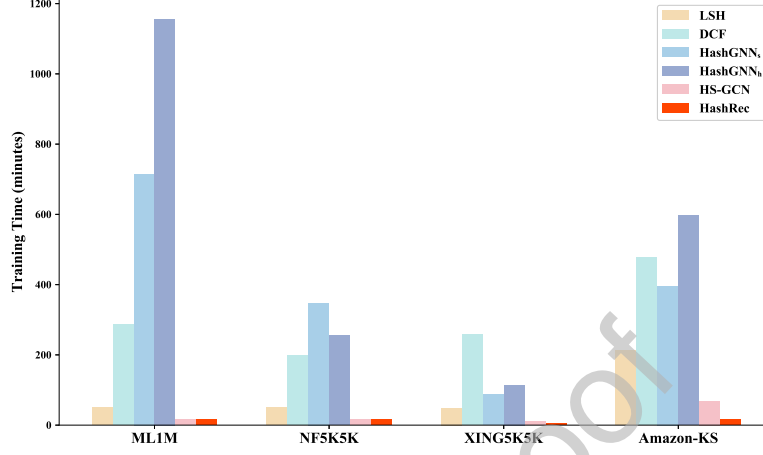


Figure 6: Time consumption comparison on different methods.

mance of our HashRec. Specifically, we adopt control variates, that is, only one hyperparameter is adjusted at a time, and the remaining parameters are kept constant, and observe the variation trends of the top-5 recommendation performance (i.e., NDCG@5 and Rec@5) on ML1M, NF5K5K, XING5K5K and Amazon-KS. We report the results in Figure 7, Figure 8 and Figure 9, and obtain the following observations.

- Effect of dimension size K . According to the experimental results shown in Figure 7, as K increases from 32 to 256, general performance improvements of our HashRec are observed on the four datasets. Specifically, (i) the recommendation performance on ML1M, XING5K5K and Amazon-KS increases monotonically, and the growth rate of performance gradually slows down as K increases from 128 to 256. (ii) the recommendation performance on NF5K5K also increases monotonically as k increases from 32 to 128, and falls back slightly at $K = 256$ after peaking at $K = 128$. The above experimental results show that longer ternary hash codes can provide stronger representation ability to alleviate the information loss caused by discretization. However, when the dimension of the hash code increases to a certain value, the marginal revenue is weakened, or even slightly redundant.
- Effect of λ_1 . As shown in Figure 8, we experiment on a series of different values of the weight parameter for the pairwise ranking loss. Overall,

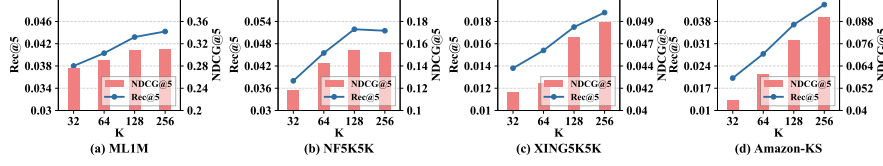
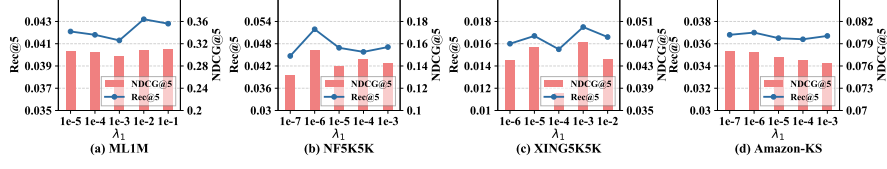
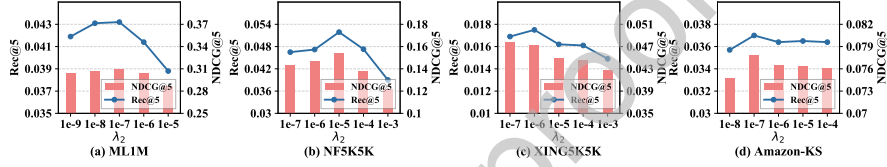


Figure 7: Impact of dimension size K on recommendation performance.

the effect of λ_1 on the model performance is non-monotonic. When λ_1 is small, the pairwise ranking supervision signal is insufficient, and it is difficult to separate the relative order of positive and negative samples. When λ_1 is large, the ranking term accounts for too high a proportion of the total loss, which will inhibit the complementary effect of other constraints on representation learning. Secondly, λ_1 exhibits dataset dependence, that is, the parameter is coupled with data sparsity and interaction distribution, but the model performance fluctuation is controllable within the effective interval. In addition, on Amazon-KS, the change of the weight parameter λ_1 has little fluctuations on the overall performance, which indicates that it is relatively insensitive to the changes in λ_1 .

- Effect of λ_2 . To evaluate the influence of the sign constraint strength on the ternary hash code, we compare different values of the weight parameter for the sign constrained loss in Figure 9. We have the following observations. (i) When λ_2 is in a small order of magnitude, strengthening the sign constraint helps to improve the representation ability of the ternary hash code. (ii) When λ_2 is continuously increased after the model performance reaches its peak, the overly strong sign constraint compresses the representation freedom, weakens the representation ability of the ternary hash code to capture the fine-grained preferences, and leads to the degradation of the model performance. Compared with λ_1 , λ_2 is more sensitive to performance changes, and therefore requires a finer-grained grid search to determine its optimal value.

Figure 8: Sensitivity analysis of the hyperparameter λ_1 .Figure 9: Sensitivity analysis of the hyperparameter λ_2 .

5.5.5. Interpretability Analysis (RQ5)

In this section, we analyze the learned ternary hash codes from the perspective of interpretability. Specifically, we adopt t-SNE (t-distributed Stochastic Neighbor Embedding) to map the high-dimensional user/item embeddings into a two-dimensional space for visualization. Since t-SNE preserves local neighborhood relationships relatively well, it helps us visually examine whether the hash codes capture meaningful semantic patterns, such as grouping users with similar preferences or items belonging to the same category, thereby improving the interpretability of the hash codes. In our experiments, we apply dimensionality reduction to both user and item embeddings for each dataset, and then perform K-Means in the two-dimensional space to evaluate the discrete representations. We show the results of t-SNE in Figure 10 and obtain the following observations. (i) On ML1M, NF5K5K, XING5K5K and Amazon-KS, t-SNE results exhibit a pattern in which separability and continuity coexist, while the dominant geometry, overlap strength, and boundary sharpness vary by dataset. (ii) For user embeddings, t-SNE of ML1M is closer to a partitioned and ring-like sector structure. Wherein, multiple directional regions are visible, indicating that the model can learn relatively stable user-group hierarchies. Meanwhile,

transitional overlap still exists at the center and sector boundaries, indicating that user tastes are not hard-discrete. t-SNE of NF5K5K shows that a high-density mixed core and one-sided prominent isolated cluster, indicating that the model can stably identify user-groups with strong behavior patterns. However, there is more overlap within the mixed core, which indicates that the ability of fine-grained division of the mainstream users is limited. t-SNE of XING5K5K has the most obvious central collapse, strongest inter-cluster overlap and weakest boundaries, which indicates that users' interests intersect more and the continuity of preferences is higher on this dataset. The hash codes generated by our model can distinguish the mainstream user categories, but they are less effective at distinguishing highly similar users. t-SNE of Amazon-KS shows that a relatively balanced pattern of mixed core and several marginal sub-groups, which indicates that the model not only preserves the continuous relationship between mainstream users and avoids over-division, but also separate users with more consistent behavior patterns and more single interest direction. (iii) For item embeddings, t-SNE of ML1M shows that the overall partition is clear and there is an independent sub-group, indicating that the model can clearly distinguish the categories of items. t-SNE of NF5K5K is consistent with that of its user embeddings, i.e., presenting a two-stage distribution with a high-density mixed core and one-sided prominent isolated cluster, which indicates that the model can distinguish items with large differences, while overlap remains for items with nearby semantics or highly similar interacted-user coverage. t-SNE of XING5K5K presents mostly central mixing and a few local sub-groups, which means that items with minor differences are compressed into shared regions (the central mixing), but there are still identifiable local semantic sub-groups. t-SNE of Amazon-KS provides the clearest separation and richest structure. Multiple clusters form clear spatial gaps with high intra-cluster compactness, with only a limited transition band near the center, indicating that the hash codes can preserve multiple major semantic patterns well on this dataset. In a summary, these t-SNE results are in the line with our expectation that the model can distinguish the core differences existing in different users/items and form multiple stable sub-groups on different datasets. However, in the high-density transition region, the fine-grained differentiation of highly similar users/items is still limited by the degree of quantization and the length of hash code, especially on XING5K5K.

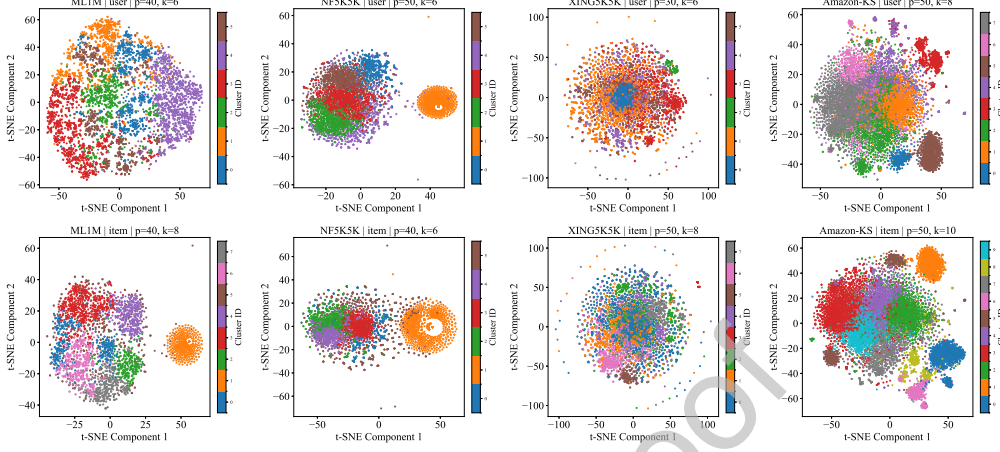


Figure 10: Interpretability Analysis (i.e., t-SNE) of all datasets.

5.5.6. Ablation Study (RQ6)

As we mentioned in Section 5.5.2, our FedHashRec differs from HashRec by only the privacy-preserving framework TriPro, and the model performance of both is comparable. Therefore, in this section, we only need to use our HashRec instead of our FedHashRec to evaluate the necessity of the model components. It should be noted that the neighborhood aggregation module is the core and defining component in a graph convolutional network, which cannot be separated. Thus, we do not consider the neighborhood aggregation module as one of the ablation studies. In this section, we perform ablation studies on the graph convolution rule, the sign constrained loss and the initial regularization term, respectively. We report the results in Table 6. After the analysis, we show our observations in the following three aspects.

- Effect of the graph convolution rule. As analyzed in Section 4.3, our graph convolution rule is theoretically effective. Now, we verify this experimentally. In Table 6, we observe that the removal of the graph convolution rule from the propagation layer of our HashRec (w/o C.R.) leads to significant performance decay on all datasets. Taking NDCG@5 as an example, the model performance of w/o C.R. is decreased by 29.19% compared to our HashRec on ML1M, 28.89% on NF5K5K, 12.71% on XING5K5K, and 15.08% on Amazon-KS. Hence, we conclude that the graph convolution rule is very important in our HashRec, which can significantly improve our ability to capture the node infor-

mation of users and items.

- Effect of the sign constrained loss. In Section 4.2.3, our analysis shows that the sign constrained loss plays an important role in the multi-objective optimization of our HashRec. The idea is that the regularization makes the initial embeddings gradually tend to be binary and improves the representation ability of the hash codes. The ablation experiments in Table 6, removing the sign constrained loss (w/o $\mathcal{L}_{\text{sign}}$), provide strong evidence for our theoretical analysis. The model performance decreases on all datasets without the sign constrained loss, confirming that this component is crucial for our HashRec.
- Effect of the initial regularization term. As mentioned in Section 4.2.3, using the initial regularization term $\|\mathbf{E}^{(0)}\|_2^2$ to optimize the discrete model would deviate from our true optimization direction. This is also confirmed by the ablation experiments (applying the initial regularization term, i.e., w/ $\mathcal{L}_{\text{ini}}$) in Table 6. We have the following two observations. (i) On ML1M, NF5K5K and Amazon-KS, using the initial regularization term makes the model performance lower than the baseline without regularization constraint (i.e., w/o $\mathcal{L}_{\text{sign}}$) in all metrics. On XING, only the NDCG@5 metric exhibits a slight increase, while all others decline. This observation again verifies that the initial regularization term is not suitable for the problem of hashing. (ii) On all datasets, our HashRec consistently outperforms the model using the initial regularization term. This comparison fully proves that the sign constrained loss we designed can guide the model optimization more effectively, and verifies the effectiveness and necessity of the component of the sign constrained loss again.

6. Conclusions and Future Work

In this paper, we study an emerging and challenging problem, i.e., how to apply hashing technology to FedGCN with a good balance among privacy protection, communication efficiency and model performance. As a response, we propose a novel privacy-preserving recommendation algorithm, i.e., Fed-HashRec, which consists of a federated framework TriPro and a hash graph convolutional method HashRec. Specifically, TriPro protects users' private information through three barriers (i.e., adaptation zone, secret zone and

Table 6: Ablation study

Dataset	Method	NDCG@5	NDCG@20	NDCG@50	Pre@5	Pre@20	Pre@50	Rec@5	Rec@20	Rec@50
ML1M	w/o C.R.	0.2188	0.1995	0.1971	0.2180	0.1817	0.1421	0.0267	0.0891	0.1679
	w/o $\mathcal{L}_{\text{sign}}$	0.2969	0.2565	0.2546	0.2878	0.2232	0.1723	0.0415	0.1211	0.2202
	w/ $\mathcal{L}_{\text{ini}}$	0.2944	0.2541	0.2508	0.2865	0.2214	0.1710	0.0405	0.1181	0.2149
	HashRec	0.3089	0.2661	0.2635	0.2977	0.2307	0.1778	0.0432	0.1239	0.2252
NF5K5K	w/o C.R.	0.1093	0.1219	0.1533	0.1071	0.0940	0.0756	0.0330	0.1158	0.2221
	w/o $\mathcal{L}_{\text{sign}}$	0.1404	0.1457	0.1762	0.1325	0.1042	0.0813	0.0458	0.1368	0.2434
	w/ $\mathcal{L}_{\text{ini}}$	0.1369	0.1444	0.1756	0.1295	0.1044	0.0817	0.0463	0.1352	0.2434
	HashRec	0.1537	0.1573	0.1877	0.1440	0.1113	0.0853	0.0519	0.1457	0.2548
XING5K5K	w/o C.R.	0.0412	0.0475	0.0605	0.0393	0.0364	0.0328	0.0133	0.0436	0.0901
	w/o $\mathcal{L}_{\text{sign}}$	0.0458	0.0529	0.0668	0.0435	0.0385	0.0340	0.0170	0.0501	0.0983
	w/ $\mathcal{L}_{\text{ini}}$	0.0465	0.0522	0.0646	0.0435	0.0379	0.0329	0.0161	0.0482	0.0941
	HashRec	0.0472	0.0533	0.0669	0.0445	0.0386	0.0337	0.0175	0.0500	0.0972
Amazon-KS	w/o C.R.	0.0661	0.0757	0.0962	0.0623	0.0451	0.0319	0.0306	0.0828	0.1380
	w/o $\mathcal{L}_{\text{sign}}$	0.0763	0.0852	0.1058	0.0713	0.0494	0.0339	0.0358	0.0915	0.1475
	w/ $\mathcal{L}_{\text{ini}}$	0.0754	0.0851	0.1056	0.0704	0.0493	0.0338	0.0356	0.0923	0.1477
	HashRec	0.0778	0.0862	0.1072	0.0727	0.0494	0.0341	0.0370	0.0925	0.1491

verification zone). Secret sharing only needs to be performed once per convolution, and then the server can obtain key information by combining the information of the three zones to learn item embedding. Moreover, we theoretically prove that the key information obtained is lossless, which allows the representation ability of the model to be preserved. In HashRec, we design a graph convolution rule, which can better capture the node information between users and items and learn high-quality hash codes. We conduct extensive experiments on four datasets to show that our FedHashRec outperforms the existing [fully discrete](#) methods, indicating that our FedHashRec can achieve high-quality recommendations while preserving privacy. In addition, to ensure the fairness and accuracy of the evaluation, we conduct time consumption experiments on HashRec, the core component of our FedHashRec. The results show that HashRec is of higher efficiency, which provides strong support for the deployment of our FedHashRec in distributed scenarios. In summary, our FedHashRec achieves high recommendation performance and potentially efficient computation while protecting user privacy, showing a wide application prospect in real-world privacy-preserving recommendation scenarios.

In the future, we are interested in exploring better gradient estimation methods rather than limited to applying STE, aiming to measure the gradient information in backpropagation more accurately, thus alleviating the gradient bias problem in the existing methods and further improving the stability and performance of a discrete model. Besides, we are also interested in designing a better graph convolution method to capture deeper relationships between

users and items.

Declaration of interests

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

7. Acknowledgement

We thank the support of Basic Research Fund in Shenzhen Natural Science Foundation (No.JCYJ20240813141441054), National Natural Science Foundation of China (Nos.62461160311, 62272315), and National Key Research and Development Program of China (No.2023YFF0725100).

References

- [1] A. Grover, J. Leskovec, node2vec: Scalable feature learning for networks, in: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD'16, 2016, pp. 855–864.
- [2] X. Wang, X. He, M. Wang, F. Feng, T.-S. Chua, Neural graph collaborative filtering, in: Proceedings of the 42nd international ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR'19, 2019, pp. 165–174.
- [3] J. Cho, D. Hyun, S. Kang, H. Yu, Learning heterogeneous temporal patterns of user preference for timely recommendation, in: Proceedings of the Web Conference 2021, WWW'21, 2021, pp. 1274–1283.
- [4] W. Hamilton, Z. Ying, J. Leskovec, Inductive representation learning on large graphs, in: Proceedings of the 30th International Conference on Neural Information Processing Systems, NeuIPS'17, 2017, pp. 1024–1034.
- [5] X. He, K. Deng, X. Wang, Y. Li, Y. Zhang, M. Wang, Lightgcn: Simplifying and powering graph convolution network for recommendation, in: Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR'20, 2020, pp. 639–648.

- [6] Z. Fan, K. Xu, Z. Dong, H. Peng, J. Zhang, P. S. Yu, Graph collaborative signals denoising and augmentation for recommendation, in: Proceedings of the 46th international ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR'23, 2023, pp. 2037–2041.
- [7] B. McMahan, E. Moore, D. Ramage, S. Hampson, B. A. y Arcas, Communication-efficient learning of deep networks from decentralized data, in: Proceedings of the 20th International Conference on Artificial Intelligence and Statistics, AISTATS'17, 2017, pp. 1273–1282.
- [8] S. Jiang, H. Yang, Q. Xie, C. Ma, S. Wang, Z. Liu, T. Xiang, G. Xing, Towards compute-efficient byzantine-robust federated learning with fully homomorphic encryption, *Nature Machine Intelligence* 7 (2025) 1–12.
- [9] J. Lou, C. Rong, H. Chen, D. Liu, Efficient federated graph aggregation for privacy-preserving gnn-based session recommendation, *Scientific Reports* 15 (1) (2025) 23394.
- [10] D. Yao, W. Pan, Y. Shi, M. J. O'Neill, Y. Dai, Y. Wan, P. Zhao, H. Jin, L. Sun, Fedhm: Efficient federated learning for heterogeneous models via low-rank factorization, *Artificial Intelligence* (2025) 104333.
- [11] C. Wu, F. Wu, L. Lyu, T. Qi, Y. Huang, X. Xie, A federated graph neural network framework for privacy-preserving personalization, *Nature Communications* 13 (1) (2022) 3091.
- [12] L. Qu, N. Tang, R. Zheng, Q. V. H. Nguyen, Z. Huang, Y. Shi, H. Yin, Semi-decentralized federated ego graph learning for recommendation, in: Proceedings of the ACM on Web Conference 2023, WWW'23, 2023, pp. 339–348.
- [13] P. M. T. Do, J. Lu, Q. Zhang, G. Zhang, A federated graph neural network with differential privacy for cross-domain recommender system, *ACM Transactions on Intelligent Systems and Technology* 16 (4) (2025).
- [14] X. Zhao, H. Gu, L. Fan, Y. Han, Q. Yang, Disentangling data distribution for optimal and communication-efficient federated learning, *Artificial Intelligence* (2025) 104455.

- [15] X. Zhang, Y. Pang, Y. Kang, W. Chen, L. Fan, H. Jin, Q. Yang, No free lunch theorem for privacy-preserving llm inference, *Artificial Intelligence* 341 (2025) 104293.
- [16] A. Gionis, P. Indyk, R. Motwani, Similarity search in high dimensions via hashing, in: *Proceedings of 25th International Conference on Very Large Data Bases, VLDB'99*, 1999, pp. 518–529.
- [17] H. Zhang, F. Shen, W. Liu, X. He, H. Luan, T.-S. Chua, Discrete collaborative filtering, in: *Proceedings of the 39th International ACM SIGIR conference on Research and Development in Information Retrieval, SIGIR'16*, 2016, pp. 325–334.
- [18] H. Liu, X. He, F. Feng, L. Nie, R. Liu, H. Zhang, Discrete factorization machines for fast feature-based recommendation, in: *Proceedings of the 27th International Joint Conference on Artificial Intelligence, IJCAI'18*, 2018, p. 3449–3455.
- [19] I. Hubara, M. Courbariaux, D. Soudry, R. El-Yaniv, Y. Bengio, Binarized neural networks, in: *Proceedings of the 29th International Conference on Neural Information Processing Systems, NeuIPS'16*, 2016, pp. 4114–4122.
- [20] Z. Cao, M. Long, J. Wang, P. S. Yu, Hashnet: Deep learning to hash by continuation, in: *Proceedings of the 16th IEEE International Conference on Computer Vision, ICCV'17*, 2017, pp. 5608–5617.
- [21] H. Liu, Y. Wei, J. Yin, L. Nie, Hs-gcn: Hamming spatial graph convolutional networks for recommendation, *IEEE Transactions on Knowledge and Data Engineering* 35 (6) (2022) 5977–5990.
- [22] Q. Tan, N. Liu, X. Zhao, H. Yang, J. Zhou, X. Hu, Learning to hash with graph neural networks for recommender systems, in: *Proceedings of the ACM on Web Conference 2020, WWW'20*, 2020, pp. 1988–1998.
- [23] Y. Chen, Y. Fang, Y. Zhang, C. Ma, Y. Hong, I. King, Towards effective top-n hamming search via bipartite graph contrastive hashing, *IEEE Transactions on Knowledge and Data Engineering* 36 (12) (2024) 9418–9432.

- [24] M. Kapralov, M. Makarov, C. Sohler, On the adversarial robustness of locality-sensitive hashing in hamming space, *Proceedings of the ACM on Management of Data* 3 (2) (2025) 1–24.
- [25] S. Bhattacharya, S. Dey, E. Goldenberg, M. Habib, B. Haeupler, M. Koucký, Constant rate isometric embeddings of hamming metric into edit metric, *CoRR* abs/2504.03605 (2025).
- [26] R. Ying, R. He, K. Chen, P. Eksombatchai, W. L. Hamilton, J. Leskovec, Graph convolutional neural networks for web-scale recommender systems, in: *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD’18*, 2018, pp. 974–983.
- [27] Y. Yang, L. Xia, D. Luo, K. Lin, C. Huang, Graphpro: Graph pre-training and prompt learning for recommendation, in: *Proceedings of the ACM Web Conference 2024, WWW’24*, 2024, pp. 3690–3699.
- [28] G. Chen, L. Xia, C. Huang, Lightgnn: Simple graph neural network for recommendation, in: *Proceedings of the 18th ACM International Conference on Web Search and Data Mining, WSDM’25*, 2025, pp. 549–558.
- [29] Y. Zhang, D. Lian, G. Yang, Discrete personalized ranking for fast collaborative filtering from implicit feedback, in: *Proceedings of the 31st AAAI Conference on Artificial Intelligence, AAAI’17*, 2017, pp. 1669–1675.
- [30] Y. Xu, L. Zhu, Z. Cheng, J. Li, J. Sun, Multi-feature discrete collaborative filtering for fast cold-start recommendation, in: *Proceedings of the 34th AAAI Conference on Artificial Intelligence, AAAI’20*, 2020, pp. 270–278.
- [31] H. Wang, D. Lian, Y. Ge, Binarized collaborative filtering with distilling graph convolutional networks, in: *Proceedings of the 28th International Joint Conference on Artificial Intelligence, IJCAI’19*, 2019, pp. 4802–4808.
- [32] W.-C. Kang, J. McAuley, Candidate generation with binary codes for large-scale top-n recommendation, in: *Proceedings of the 28th ACM*

- International Conference on Information and Knowledge Management, CIKM'19, 2019, pp. 1523–1532.
- [33] Y. Chen, Y. Fang, Y. Zhang, I. King, Bipartite graph convolutional hashing for effective and efficient top-n search in hamming space, in: Proceedings of the ACM on Web Conference 2023, WWW'23, 2023, pp. 3164–3172.
 - [34] X. Wu, D. Loveland, R. Chen, Y. Liu, X. Chen, L. Neves, A. Jadbabaie, M. Ju, N. Shah, T. Zhao, Graphhash: Graph clustering enables parameter efficiency in recommender systems, in: Proceedings of the ACM on Web Conference 2025, WWW'25, 2025, pp. 357–369.
 - [35] M. Ammad-Ud-Din, E. Ivannikova, S. A. Khan, W. Oyomno, Q. Fu, K. E. Tan, A. Flanagan, Federated collaborative filtering for privacy-preserving personalized recommendation system, CoRR abs/1901.09888 (2019).
 - [36] G. Lin, F. Liang, W. Pan, Z. Ming, Fedrec: Federated recommendation with explicit feedback, IEEE Intelligent Systems 36 (5) (2020) 21–30.
 - [37] Z. Lin, W. Pan, Q. Yang, Z. Ming, A generic federated recommendation framework via fake marks and secret sharing, ACM Transactions on Information Systems 41 (2) (2022) 1–37.
 - [38] J. Zeng, D2pdrf: A differential privacy recommendation algorithm based on federated learning, in: Proceedings of the 3rd International Symposium on Computer Applications and Information Systems, ISCAIS'2024, 2024, pp. 460–466.
 - [39] N.-H. Nguyen, T.-A. Nguyen, T. Nguyen, V. T. Hoang, D. D. Le, K.-S. Wong, Towards efficient communication and secure federated recommendation system via low-rank training, in: Proceedings of the ACM on Web Conference 2024, WWW'24, 2024, pp. 3940–3951.
 - [40] Z. Li, G. Long, T. Zhou, J. Jiang, C. Zhang, Personalized federated collaborative filtering: A variational autoencoder approach, in: Proceedings of the 39th AAAI Conference on Artificial Intelligence, AAAI'25, 2025, pp. 18602–18610.

- [41] L. Qu, J. Li, W. Yuan, P. Ruan, Y. Shi, H. Yin, Proxy model-guided reinforcement learning for client selection in federated recommendation, CoRR abs/2508.10401 (2025).
- [42] P. Liu, G. Lu, Addressing data imbalance for federated recommender systems: a rebalancing framework with gradient alignment regularization, *Journal of Intelligent Information Systems* 63 (2) (2025) 657–679.
- [43] L. Shen, A. Li, X. Liu, R. Xia, B. Yang, Grouping-enhanced personalization for federated recommendation, *Information Processing and Management* 63 (4) (2026) 104599.
- [44] R. Ahlswede, L. H. Khachatrian, The diametric theorem in hamming spaces—optimal anticodes, *Advances in Applied mathematics* 20 (4) (1998) 429–449.
- [45] M. J. Mia, M. H. Amini, Quancrypt-fl: Quantized homomorphic encryption with pruning for secure federated learning, *IEEE Transactions on Artificial Intelligence* (2025) 1–16.
- [46] Z. Xu, C. Chu, S. Song, An effective federated recommendation framework with differential privacy, *Electronics* 13 (8) (2024) 1589.
- [47] K. Chehbouni, M. De Cock, G. Caporossi, A. Taik, R. Rabbany, G. Farnadi, Enhancing privacy in the early detection of sexual predators through federated learning and differential privacy, in: *Proceedings of the 39th AAAI Conference on Artificial Intelligence, AAAI’25, 2025*, pp. 27887–27895.
- [48] J. Qin, X. Zhang, B. Liu, J. Qian, A split-federated learning and edge-cloud based efficient and privacy-preserving large-scale item recommendation model, *Journal of Cloud Computing* 12 (1) (2023) 57.
- [49] W. Pan, L. Chen, Gbpr: Group preference based bayesian personalized ranking for one-class collaborative filtering., in: *Proceedings of the 23rd International Joint Conference on Artificial Intelligence, IJCAI’13, 2013*, pp. 2691–2697.
- [50] P. Hu, Z. Lin, W. Pan, Q. Yang, X. Peng, Z. Ming, Privacy-preserving graph convolution network for federated item recommendation, *Artificial Intelligence* 324 (2023) 103996.

- [51] Z. Xian, E. Yang, W. Pan, Z. Ming, Fedhog: Federated homogeneous graph neural network for privacy-preserving recommendation, ACM Transactions on Information Systems 44 (3) (2026) 1–34.

Journal Pre-proof